

EDA 详细教程（学生版）

面向学生的 **step-by-step** 详细教程。本教程基于代码文件 `src/01_eda_tutorial.py`，按模块逐行解释每个函数和参数的含义，配合实际运行结果与图示。

教程模块：

- 模块 0：环境准备与数据加载
- 模块 1：数据概况
- 模块 2：缺失值分析
- 模块 3：分布分析
- 模块 4：离群值分析

模块 0：环境准备与数据加载

本模块是整个 EDA（探索性数据分析）教程的起点。在开始任何数据分析工作之前，我们必须先把“地基”打好：导入正确的库、配置好路径、把数据干净地加载进内存，并处理好目标变量的缺失问题。本模块对应代码文件 `src/01_eda_tutorial.py` 的第 19–72 行。

学习目标

学完本模块后，你将能够：

1. **理解每个 import 语句的作用**：知道 pandas、numpy、matplotlib、seaborn、scipy 等核心库分别在数据分析中扮演什么角色。
2. **掌握路径配置的最佳实践**：理解为什么用 `os.path.join()` 而不是字符串拼接，以及 `os.makedirs(..., exist_ok=True)` 的含义。
3. **正确读取 CSV 文件**：理解 `pd.read_csv()` 中 `low_memory=False` 和 `encoding='latin-1'` 这两个关键参数的作用。
4. **处理目标变量的缺失值**：学会用 `dropna(subset=...)` 删除特定列缺失的样本，并理解 `reset_index(drop=True)` 的意义。
5. **掌握布尔转整型的技巧**：理解 `(df['col'] == 'value').astype(int)` 如何把字符串标签转换成 0/1 数值。
6. **理解为什么 Status.Vital 缺失率高达 88%**：从数据采集机制层面理解缺失原因，并明白为什么不能简单地把缺失当作死亡。
7. **评估数据集的内存占用**：学会用 `df.memory_usage(deep=True)` 监控内存。

一、导入必要的库

数据分析的第一步永远是导入所需的库。下面这段代码完成了所有库的导入：

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
import warnings
from datetime import datetime
from scipy import stats
```

1.1 逐行解释每个 import

import pandas as pd

pandas 是 Python 中最核心的数据分析库，它提供了 **DataFrame**（数据框）这种二维表格结构，类似于 Excel 表格或 SQL 表。几乎所有数据加载、清洗、变换操作都依赖它。

- **as pd** 是给库起一个简短的别名，这样我们写 **pd.read_csv()** 而不是 **pandas.read_csv()**，更简洁。
- 常用功能：**read_csv()** 读数据、**DataFrame** 数据结构、**dropna()** 删缺失值、**value_counts()** 统计频数、**groupby()** 分组聚合等。

import numpy as np

NumPy (Numerical Python) 是 Python 数值计算的基石，提供了高效的多维数组 **ndarray** 和大量数学函数。**pandas** 底层就是基于 **numpy** 构建的。

- **as np** 同样是别名约定，整个 Python 社区都遵守这个约定。
- 常用功能：**np.nan**（缺失值表示）、**np.mean()**、**np.std()**、数组运算等。
- 在本教程中，**numpy** 主要用于数值计算和数组操作。

import matplotlib.pyplot as plt

matplotlib 是 Python 最基础的可视化库，**pyplot** 是它的子模块，提供了类似 MATLAB 的绘图接口。

- **as plt** 是社区约定的别名。
- 常用功能：**plt.figure()**、**plt.subplot()**、**plt.savefig()**、**plt.close()** 等。
- 在本教程中，所有图表（条形图、饼图、直方图等）都用 **matplotlib** 绘制。

import seaborn as sns

seaborn 是基于 **matplotlib** 的高级可视化库，提供了更美观的默认样式和更简洁的 API，特别擅长统计图表。

- **as sns** 是社区约定别名（**seaborn** 名字来源于美剧《西部世界》中的角色）。
- 常用功能：**sns.heatmap()**（热力图）、**sns.boxplot()**（箱线图）、**sns.distplot()**（分布图）等。
- 虽然本模块代码没有直接调用 **seaborn**，但后续模块会大量使用，所以提前导入。

import os

os 是 Python 标准库，提供了与操作系统交互的功能，比如路径操作、目录创建、环境变量等。

- 不需要安装，Python 自带。
- 常用功能：**os.path.join()**（拼接路径）、**os.makedirs()**（创建目录）、**os.path.exists()**（判断路径是否存在）。

import warnings

warnings 是 Python 标准库，用于控制警告信息的输出。

- 不需要安装，Python 自带。
- 常用功能：**warnings.filterwarnings()**（过滤警告）。

```
from datetime import datetime
```

`datetime` 是 Python 标准库中处理日期时间的模块。`from datetime import datetime` 这种写法表示从 `datetime` 模块中只导入 `datetime` 这个类。

- 常用功能：`datetime.now()`（获取当前时间）、时间格式化、时间差计算等。
- 在本教程中主要用于记录脚本运行时间戳。

```
from scipy import stats
```

SciPy (Scientific Python) 是建立在 `numpy` 之上的科学计算库，`stats` 子模块提供了大量统计函数。

- `from scipy import stats` 表示只导入 `stats` 子模块。
- 常用功能：`stats.shapiro()`（正态性检验）、`stats.ttest_ind()`（t 检验）、`stats.mannwhitneyu()`（Mann-Whitney U 检验）、`stats.zscore()`（Z 分数）等。
- 在后续的统计分析和离群值检测模块中会大量使用。

小贴士：导入库时，`import xxx as yyy` 中的别名是社区约定俗成的，建议遵守这些约定（如 `pd`、`np`、`plt`、`sns`），这样别人看你的代码会更容易理解。

二、关闭警告信息

```
warnings.filterwarnings('ignore')
```

2.1 这行代码的作用

`warnings.filterwarnings('ignore')` 的作用是**关闭所有警告信息的输出**。在运行数据分析脚本时，`pandas`、`numpy`、`matplotlib` 等库可能会输出大量警告，例如：

- `FutureWarning`：某个函数在未来的版本中会改变行为。
- `DeprecationWarning`：某个函数已经被弃用。
- `RuntimeWarning`：运行时的一些非致命问题（比如除以零、无效值等）。

2.2 为什么要关闭警告

在**教学脚本**和**探索性分析**阶段，关闭警告有几个好处：

1. **保持输出干净**：让控制台输出聚焦于真正的结果，不被大量警告淹没。
2. **避免初学者困惑**：初学者看到一堆英文警告容易误以为代码出错了。
3. **教学聚焦**：教学时希望学生关注核心逻辑，而不是被次要信息干扰。

常见问题：在生产环境或正式项目中，**不建议**无脑关闭所有警告。警告往往提示潜在问题（比如某个函数即将废弃、数据类型不匹配等），应该认真对待。更好的做法是针对性地过滤特定警告，例如：

```
python warnings.filterwarnings('ignore', category=FutureWarning)
```

三、路径配置

```
BASE_DIR = "/home/wjj/Documents/trae_projects/ml_template"
DATA_PATH = os.path.join(BASE_DIR, "data", "cancer_data_eng.csv")
IMG_DIR = os.path.join(BASE_DIR, "img")
RESULTS_DIR = os.path.join(BASE_DIR, "results")
```

3.1 路径配置的整体思路

这段代码定义了四个关键路径变量：

变量名	含义	指向
<code>BASE_DIR</code>	项目根目录	<code>/home/wjj/Documents/trae_projects/ml_template</code>
<code>DATA_PATH</code>	数据文件路径	<code>data/cancer_data_eng.csv</code>
<code>IMG_DIR</code>	图片输出目录	<code>img/</code>
<code>RESULTS_DIR</code>	文本结果输出目录	<code>results/</code>

这种"先定义根目录，再拼接子路径"的方式有两个好处：

1. **可维护性**：如果项目搬家，只需要改 `BASE_DIR` 一处，其他路径自动更新。
2. **可读性**：路径的语义结构清晰，一眼就能看出每个文件属于哪个目录。

3.2 `os.path.join()` 的作用

`os.path.join()` 是 Python 中拼接路径的**标准做法**，它会把多个字符串用正确的路径分隔符连接起来。

```
os.path.join(BASE_DIR, "data", "cancer_data_eng.csv")
# 在 Linux/Mac 上得到：
# /home/wjj/Documents/trae_projects/ml_template/data/cancer_data_eng.csv
# 在 Windows 上得到：
# C:\...\ml_template\data\cancer_data_eng.csv
```

3.3 为什么用 `os.path.join()` 而不是字符串拼接

你可能会想，为什么不直接写成 `BASE_DIR + "/data/cancer_data_eng.csv"`？原因有三：

1. **跨平台兼容**：不同操作系统的路径分隔符不同（Linux/Mac 用 `/`，Windows 用 `\`）。`os.path.join()` 会自动选择当前系统正确的分隔符，而字符串拼接写死 `/` 在 Windows 上可能出问题。
2. **避免分隔符错误**：如果 `BASE_DIR` 末尾已经有 `/`，字符串拼接会变成 `//`（虽然大多数系统能容忍，但不规范）；`os.path.join()` 会自动处理这种情况。
3. **可读性和规范性**：这是 Python 社区的最佳实践，所有教程和项目都推荐这样做。

小贴士：在 Python 3.4+ 中，还可以使用 `pathlib.Path` 来处理路径，它更加面向对象，例如：

```
python from pathlib import Path BASE_DIR = Path("/home/wjj/Documents/trae_projects/ml_template") DATA_PATH = BASE_DIR / "data" / "cancer_data_eng.csv"
不过本教程为了兼容性和简洁性，仍然使用传统的 os.path.join()。
```

四、创建输出目录

```
os.makedirs(IMG_DIR, exist_ok=True)
os.makedirs(RESULTS_DIR, exist_ok=True)
```

4.1 `os.makedirs()` 的作用

`os.makedirs()` 用于**递归创建多级目录**。比如 `IMG_DIR` 是 `/home/wjj/Documents/trae_projects/ml_template/img`，如果 `ml_template` 目录都不存在，`os.makedirs()` 会一次性把所有缺失的父目录都创建出来。

与之相对的是 `os.mkdir()`，它只能创建一级目录，如果父目录不存在会报错。

4.2 exist_ok=True 参数的含义

`exist_ok` 是 `os.makedirs()` 的一个关键字参数，含义是“如果目录已存在，是否允许（OK）”。

- `exist_ok=True`（本教程使用）：如果目录已经存在，**不报错**，直接跳过。
- `exist_ok=False`（默认值）：如果目录已经存在，会抛出 `FileExistsError` 异常。

小贴士：在数据分析脚本中，推荐始终使用 `exist_ok=True`，因为脚本可能被多次运行，目录很可能已经存在，不希望每次都因为目录已存在而报错中断。

五、打印教程标题

```
print("=" * 70)
print("案例教程 1：数据探索分析（EDA）")
print("=" * 70)
```

这段代码用 `=` 重复 70 次作为分隔线，打印出教程标题。这是一种常见的控制台输出美化方式，让输出结果更有层次感。

- `"=" * 70` 是 Python 字符串乘法，表示把 `=` 重复 70 次。
- 这种分隔线在长脚本中能帮助区分不同模块的输出。

六、加载数据

```
print("\n[0] 加载数据...")
df = pd.read_csv(DATA_PATH, low_memory=False, encoding='latin-1')
print(f"    原始数据集形状：{df.shape}")
```

6.1 pd.read_csv() 函数详解

`pd.read_csv()` 是 pandas 读取 CSV 文件的核心函数。本教程使用了两个关键参数：`low_memory=False` 和 `encoding='latin-1'`。

参数 1： `low_memory=False`

默认情况：pandas 读取 CSV 时默认 `low_memory=True`，它会分块读取文件，逐块推断每列的数据类型。这种方式内存友好，但有一个问题——如果某一列在前几行是整数，到后面突然变成字符串，pandas 会发出警告：

```
DtypeWarning: Columns (X, Y) have mixed types.
```

设置 `low_memory=False`：pandas 会**一次性读取整个文件**，然后统一推断每列的数据类型。这样不会出现类型推断不一致的问题，但代价是内存占用更高。

为什么本教程要这样设置：

- 本数据集有 38 个字段、约 178 万条记录，字段类型复杂，部分列确实存在混合类型。
- 为了避免 `DtypeWarning` 干扰教学，统一关闭分块读取。
- 数据集虽然大，但服务器内存足够，可以承受一次性加载。

常见问题：如果你的数据集非常大（比如几个 GB），设置 `low_memory=False` 可能导致内存溢出（OOM）。这时应该改用分块读取：

```
python chunk_iter = pd.read_csv(DATA_PATH, chunksize=100000) for chunk in chunk_iter: # 逐块处理
    process(chunk)
```

参数 2: `encoding='latin-1'`

`encoding` 参数指定文件的字符编码。常见的编码有：

- `utf-8`：最通用的编码，支持多语言。
- `latin-1`（也叫 `iso-8859-1`）：西欧语言编码，能表示 0-255 的所有字节。
- `gbk`：简体中文编码。

为什么本教程用 `latin-1`：

- 本数据集来自巴西，包含葡萄牙语字符（如 `ç`、`ã`、`õ`、`é` 等），这些字符在 `latin-1` 编码下能正确表示。
- 如果用默认的 `utf-8` 读取，遇到无法解码的字节会抛出 `UnicodeDecodeError`。
- `latin-1` 的一个特性是：**任何字节序列都能被它解码**（因为它定义了 0-255 的所有字节），所以不会报错。代价是某些字符可能显示不正确，但至少能读进来。

小贴士：遇到 `UnicodeDecodeError` 时，可以尝试以下几种编码：

1. 先试 `utf-8`（最通用）
2. 再试 `latin-1`（万能兜底，不会报错）
3. 如果是中文数据，试 `gbk` 或 `gb2312`
4. 用 `chardet` 库自动检测编码：`import chardet; chardet.detect(open(file, 'rb').read(10000))`

6.2 `df.shape` 的含义

`df.shape` 返回一个元组（行数，列数），表示数据框的形状。

```
print(f"    原始数据集形状: {df.shape}")
# 输出类似: 原始数据集形状: (1778176, 38)
```

- `df.shape[0]` 是行数（样本数）
- `df.shape[1]` 是列数（特征数）

七、处理目标变量的缺失值

这是本模块**最关键**的一步，也是理解整个数据集的关键。

7.1 背景：Status.Vital 的高缺失率

```
# 处理 Status.Vital（目标变量）的缺失值
n_before = len(df)
df = df.dropna(subset=['Status.Vital']).reset_index(drop=True)
n_after = len(df)
print(f"    删除 Status.Vital 缺失样本: {n_before:,} → {n_after:,} (删除 {n_before - n_after:,} 条, {(n_before - n_after) / n_before * 100}% 缺失率")
```

`Status.Vital` 是本数据集的**目标变量**（label），表示患者的生死状态：

- `VIVO`（葡萄牙语“活着”）→ 存活
- `MORTO`（葡萄牙语“死亡”）→ 死亡
- 缺失 → 未知

问题：这个变量的缺失率高达 **88.20%**（1,568,418 条记录没有明确的生死标签）！

7.2 为什么缺失率这么高？4 个原因

在癌症登记数据库中，`Status.Vital` 高缺失率是常见现象，主要有 4 个原因：

1. **患者刚入库，尚未到随访时间**：癌症登记系统会持续随访患者，但新入库的患者还没到随访节点，所以 `Status.Vital` 暂时为空。这部分患者其实可能还活着，只是还没确认。
2. **患者失访 (Lost to follow-up)**：有些患者搬走了、换电话了、或者不再回医院复诊，登记中心联系不上他们，无法确认生死状态。这类缺失反映了真实世界的数据采集困难。
3. **数据采集不完整**：登记中心的工作人员可能漏填、错填，或者数据录入系统有 bug，导致部分记录的 `Status.Vital` 字段为空。
4. **该变量本来只对部分患者记录**：在某些登记流程中，`Status.Vital` 只在特定时间点（比如年度复核）才更新，平时不强制填写，所以很多历史记录没有这个字段。

7.3 为什么不能把缺失当作死亡？

这是一个**非常容易犯的错误**！初学者可能会想："既然缺失了，那大概率是死了吧？"——**这种想法是危险的**，原因如下：

1. **缺失并非随机 (非 MCAR)**：从上面的 4 个原因可以看出，缺失往往和"刚入库"、"失访"等状态相关，并不是随机发生的。如果把缺失都归为死亡，会**严重高估死亡率**。
2. **会严重低估真实存活率**：本数据集中 88% 的样本缺失，如果把它们都算作死亡，那存活率会被压到极低（不到 5%），这显然不符合癌症患者的真实生存情况。
3. **会引入系统性偏差**：缺失样本和有标签样本在特征分布上可能不同（比如缺失样本可能更年轻、更早期），把它们当作死亡会让模型学到错误的模式。
4. **违背统计原则**：在统计学中，"缺失"和"死亡"是两个完全不同的概念。把未知当作已知，是一种"数据造假"。

重要概念：在统计学中，缺失机制分为三类：

- **MCAR (Missing Completely At Random)**：完全随机缺失，缺失和任何变量都无关。
- **MAR (Missing At Random)**：随机缺失，缺失只和已观测的变量有关。
- **MNAR (Missing Not At Random)**：非随机缺失，缺失和未观测的值本身有关。

本数据集的 `Status.Vital` 缺失更接近 **MNAR**（因为缺失和患者是否失访有关，而失访本身又和生死状态相关），所以不能简单处理。

7.4 处理方式：删除缺失样本

本教程采取的处理方式是：**删除 `Status.Vital` 缺失的样本，仅保留有明确生死标签的记录。**

```
df = df.dropna(subset=['Status.Vital']).reset_index(drop=True)
```

处理后的数据：

- 原始：1,778,176 条
- 删除：1,568,418 条 (88.20%)
- 保留：209,758 条

7.5 `dropna(subset=['Status.Vital'])` 详解

`dropna()` 是 pandas 删除缺失值的方法，`subset` 参数指定**只检查哪些列**。

- `subset=['Status.Vital']`：只检查 `Status.Vital` 这一列，如果这一列是 `NaN`，就删除整行。
- 如果不指定 `subset`，`dropna()` 会删除**任何一列**有缺失的行，这在有很多列的数据集上会删掉几乎所有数据。

小贴士： `subset` 参数非常有用。实际数据中，几乎每行都或多或少有缺失值，如果无脑 `dropna()` 会把数据删光。用 `subset` 可以精确控制只针对关键列（比如目标变量）删除。

7.6 `reset_index(drop=True)` 详解

删除行之后，数据框的索引（index）会保留原来的值，出现"空洞"。比如删除了第 3、5、7 行后，索引变成 `0, 1, 2, 4, 6, 8, ...`，不连续。

`reset_index()` 会重新设置索引，让索引从 0 开始连续递增。`drop` 参数的含义：

- `drop=True`：丢弃原来的索引，不把它作为新列保留。
- `drop=False`（默认）：把原来的索引作为新的一列（叫 `index`）保留下来。

本教程用 `drop=True`，因为我们不需要保留原来的索引（原来的索引只是行号，没有信息价值）。

常见问题：为什么要 `reset_index`？因为不连续的索引在后续操作中可能引发问题：

- 用 `loc[]` 按索引取值时容易出错。
- 用 `for i in range(len(df))` 遍历时，`df.loc[i]` 可能找不到。
- 一些函数（比如 `iloc[]` 和 `loc[]`）的行为会让人困惑。

删除行后养成 `reset_index(drop=True)` 的习惯是好做法。

八、创建目标变量（布尔转整型技巧）

```
# 创建目标变量：VIVO=1 (alive), MORTO=0 (dead)
df['target'] = (df['Status.Vital'] == 'VIVO').astype(int)
```

8.1 这行代码做了什么

它创建了一个新列 `target`，把字符串标签转换成数值：

Status.Vital	(== 'VIVO')	.astype(int)	target
VIVO	True	1	1
MORTO	False	0	0

8.2 拆解这个技巧

第一步： `df['Status.Vital'] == 'VIVO'`

这是一个**比较运算**，会对 `Status.Vital` 列的每个值判断是否等于 `'VIVO'`，返回一个**布尔 Series**（True/False 序列）。

Status.Vital	== 'VIVO'	→	结果
VIVO	True		True
MORTO	True		False
VIVO	True		True
MORTO	True		False

第二步： `.astype(int)`

`astype(int)` 把布尔类型转换成整数类型。在 Python 中：

- `True` → 1
- `False` → 0

所以 `True` 变成 1, `False` 变成 0。

8.3 为什么不直接用 `if-else` ?

初学者可能会想用 `apply` + `if-else` :

```
# 不推荐的写法 (慢且啰嗦)
df['target'] = df['Status.Vital'].apply(lambda x: 1 if x == 'VIV0' else 0)
```

相比之下, `(df['Status.Vital'] == 'VIV0').astype(int)` 有两个优势:

1. **向量化操作, 速度快**: pandas 的比较运算和类型转换都是底层 C 实现的向量化操作, 比 Python 层的 `apply` + `lambda` 快几个数量级。
2. **代码简洁**: 一行搞定, 可读性更强。

小贴士: 在 pandas 中, **能用向量化操作就不用 `apply`**。向量化操作利用了底层的 C/numpy 优化, 性能远超 Python 循环。常见的向量化技巧:

- 比较运算: `df['col'] == value`
- 算术运算: `df['col'] * 2`
- 字符串方法: `df['col'].str.upper()`
- 类型转换: `df['col'].astype(int)`

8.4 为什么 `target` 用 1 表示存活、0 表示死亡?

这是一种约定, 在二分类问题中:

- **1 (正类, positive)**: 通常表示我们"关注"的事件。在医学场景中, "存活"是我们希望预测和促进的结果, 所以设为 1。
- **0 (负类, negative)**: 表示另一个类别。

注意: 不同场景下正类的定义可能不同。比如在"预测癌症"任务中, "患癌"通常设为 1 (因为我们关注的是找出患者)。在本教程中, 因为关注的是"存活预测", 所以"存活"设为 1。务必根据具体任务确定正类定义, 这会影响后续的评估指标 (如 Precision、Recall) 解读。

九、打印处理后数据集信息

```
print(f"    处理后数据集形状: {df.shape}")
print(f"    内存占用: {df.memory_usage(deep=True).sum() / 1024**2:.2f} MB")
```

9.1 `df.memory_usage(deep=True)` 详解

`df.memory_usage()` 返回每列的内存占用 (字节数), `deep` 参数的含义:

- `deep=False` (默认): 只估算内存, 对于 object 类型的列 (通常是字符串) 只算指针大小, **严重低估** 实际内存。
- `deep=True`: **深入计算** object 列中每个字符串的实际内存占用, 结果更准确但计算更慢。

`.sum()` 把所有列的内存加起来, `/ 1024**2` 把字节转换成 MB (1 MB = 1024 × 1024 字节)。

9.2 为什么要关注内存占用

- **大数据集**: 本数据集有 178 万条记录, 内存占用可能达到几百 MB 甚至几个 GB。
- **性能优化**: 如果内存占用过高, 可以考虑:
- 把 `object` 类型转成 `category` 类型 (对于取值有限的分类变量)。

- 把 `float64` 转成 `float32`（精度足够时）。
- 把 `int64` 转成 `int32` 或 `int16`。
- **避免 OOM**：监控内存可以提前发现内存不足的风险。

小贴士：用 `df.info(memory_usage='deep')` 可以一次性查看所有列的数据类型和总内存占用，非常方便。

十、本模块运行结果

运行本模块代码后，控制台输出大致如下：

```
=====
案例教程 1：数据探索分析（EDA）
=====

[0] 加载数据...
原始数据集形状：(1778176, 38)
删除 Status.Vital 缺失样本：1,778,176 → 209,758（删除 1,568,418 条，88.20%）
处理后数据集形状：(209758, 39)
内存占用：245.67 MB
```

关键数字解读

指标	数值	说明
原始样本数	1,778,176	约 178 万条记录
原始特征数	38	38 个原始字段
删除样本数	1,568,418	Status.Vital 缺失的样本
删除比例	88.20%	缺失率非常高
处理后样本数	209,758	保留有明确标签的记录
处理后特征数	39	38 个原始字段 + 1 个新建的 target
内存占用	~245 MB	处理后数据集的内存

常见问题：为什么处理后特征数从 38 变成 39？

因为我们新建了 `target` 列（`df['target'] = ...`），它是从 `Status.Vital` 派生出来的。所以处理后数据集有 38 个原始字段 + 1 个 `target` = 39 列。注意 `Status.Vital` 本身没有被删除，它仍然保留在数据集中。

十一、本模块小结

核心知识点回顾

1. **库导入**：pandas（数据处理）、numpy（数值计算）、matplotlib/seaborn（可视化）、scipy（统计）、os（路径）、warnings（警告控制）。
2. **路径管理**：用 `os.path.join()` 拼接路径，跨平台兼容；用 `os.makedirs(exist_ok=True)` 创建目录，避免重复创建报错。
3. **CSV 读取**：`low_memory=False` 一次性读取避免类型推断警告；`encoding='latin-1'` 处理葡萄牙语字符。
4. **缺失值处理**：`dropna(subset=['col'])` 只针对关键列删除；`reset_index(drop=True)` 重置索引。

5. **标签编码**: `(df['col'] == 'value').astype(int)` 是向量化布尔转整型的优雅技巧。
6. **目标变量缺失的本质**: Status.Vital 88% 缺失不是数据质量问题, 而是癌症登记系统的固有特征 (随访机制、失访、采集流程)。不能把缺失当作死亡, 否则会引入严重偏差。

下一模块预告

本模块完成了数据加载和目标变量处理, 得到了一个干净的、有明确标签的数据集 (209,758 条记录, 39 列)。下一模块 (**模块 1: 数据概况**) 将深入分析这个数据集的:

- 样本数量和特征数量
- 标签分布 (存活 vs 死亡的比例)
- 数据平衡性判断
- 可视化标签分布

思考题:

1. 如果你的服务器内存只有 2GB, 而数据集有 5GB, 你会怎么修改本模块的代码?
2. 除了删除缺失样本, 还有哪些处理目标变量缺失的方法? 各有什么优缺点?
3. 为什么 `Status.Vital` 的缺失更接近 MNAR 而不是 MCAR? 请结合 4 个缺失原因分析。

模块 1: 数据概况

本模块是 EDA (探索性数据分析) 的第一步实质性分析。在模块 0 中我们已经加载并清洗了数据, 现在要回答几个最基本的问题: 数据集有多大? 有多少特征? 标签分布如何? 数据是否平衡? 这些问题的答案将决定后续所有建模决策。本模块对应代码文件 `src/01_edu_tutorial.py` 的第 74-156 行。

学习目标

学完本模块后, 你将能够:

1. **理解数据框形状属性**: 掌握 `df.shape` 的返回值, 区分 `df.shape[0]` 和 `df.shape[1]`。
2. **熟练使用 `value_counts()`**: 理解 `normalize=True` 参数, 能够同时获取频数和比例。
3. **掌握字典取值技巧**: 理解 `.get(key, default)` 的作用, 避免 `KeyError`。
4. **精通 f-string 格式化**: 掌握 `{:,}` (千分位)、`{:>10}` (右对齐)、`{:.2f}` (小数位) 等格式化语法。
5. **判断数据平衡性**: 理解 0.8-1.2、0.5-0.8/1.2-2.0、其他 这三档分类的逻辑。
6. **用 matplotlib 绘制条形图和饼图**: 理解 `subplots`、`bar`、`pie`、`text` 等函数的每个参数。
7. **美化图表**: 学会隐藏多余边框 (`spines`)、调整布局 (`tight_layout`)、设置 DPI。
8. **正确保存和关闭图形**: 理解 `bbox_inches='tight'`、`dpi=150`、`plt.close()` 的作用。

一、数据集基本形状

```
n_samples = len(df)
n_features = df.shape[1]
print(f"\n ▶ 样本数量 (Number of samples): {n_samples:,}")
print(f" ▶ 特征数量 (Number of features): {n_features}")
print(f" ▶ 其中包含 {n_features - 1} 个原始特征 + 1 个目标变量 (target)")
```

1.1 df.shape 返回什么

`df.shape` 是 pandas DataFrame 的一个属性，返回一个元组 (tuple)，格式是 (行数, 列数)。

```
df.shape
# 返回: (209758, 39)
```

- `df.shape[0]` → 行数 (样本数)，即 209758
- `df.shape[1]` → 列数 (特征数)，即 39

1.2 len(df) VS df.shape[0]

获取行数有两种常见写法：

- `len(df)` → 返回行数，更简洁，Python 风格。
- `df.shape[0]` → 返回行数，更明确表示"形状的第 0 维"。

两者结果完全相同，本教程混用是为了展示不同写法。一般推荐 `len(df)` 用于行数，`df.shape[1]` 用于列数。

1.3 为什么特征数是 39

在模块 0 中，原始数据集有 38 个字段，我们新建了 `target` 列，所以处理后是 39 列。代码中特意强调：

```
其中包含 38 个原始特征 + 1 个目标变量 (target)
```

这是为了提醒读者：`target` 不是原始特征，而是我们从 `Status.Vital` 派生出来的标签列，**不能当作特征用于建模**（否则会造成数据泄露）。

重要概念：在机器学习中，**特征 (feature)** 和 **目标变量 (target/label)** 必须严格区分：

- 特征：用于预测的输入变量 (X)
- 目标：要预测的输出变量 (y)

如果不小心把 `target` 当作特征放进模型，模型会"作弊"地学到完美答案，但在实际预测时完全失效。这就是所谓的**数据泄露 (data leakage)**。

1.4 f-string 格式化: {n_samples:,}

`f"..."` 是 Python 3.6+ 的**格式化字符串** (f-string)，可以在字符串中直接嵌入变量和表达式。

`{n_samples:,}` 中的 `:` 是**千分位分隔符**格式化：

```
n_samples = 209758
print(f"{n_samples:,}") # 输出: 209,758
print(f"{n_samples}")   # 输出: 209758
```

加了 `:` 后，数字会从右往左每三位加一个逗号，提升可读性。对于大数字（如 1,778,176）特别有用。

小贴士：f-string 是 Python 3.6+ 引入的，比旧的 `%` 格式化和 `str.format()` 都更简洁高效。建议优先使用 f-string。

二、标签分布统计

```
# 标签分布
target_counts = df['target'].value_counts()
target_props = df['target'].value_counts(normalize=True) * 100

print(f"\n ▶ 标签分布 (Target Distribution):")
print(f"    存活 (VIVO = 1): {target_counts.get(1, 0):>10,}  ({target_props.get(1, 0):.2f}%)")
print(f"    死亡 (MORTO = 0): {target_counts.get(0, 0):>10,}  ({target_props.get(0, 0):.2f}%)")
```

2.1 value_counts() 方法详解

`value_counts()` 是 pandas Series 的方法，用于统计每个唯一值的出现次数。它返回一个新的 Series，索引是唯一值，值是频数。

```
df['target'].value_counts()
# 输出:
# 0    123862
# 1     85896
# Name: target, dtype: int64
```

默认行为:

- 按**频数降序**排列。
- 只统计非缺失值 (`NaN` 会被忽略)。
- 返回的 Series 索引是原 Series 的唯一值。

2.2 normalize=True 参数

`normalize=True` 让 `value_counts()` 返回**比例**而不是频数:

```
df['target'].value_counts(normalize=True)
# 输出:
# 0    0.590479
# 1    0.409521
# Name: target, dtype: float64
```

- `normalize=False` (默认): 返回频数 (整数)。
- `normalize=True`: 返回比例 (小数, 总和为 1)。

本教程中 `target_props = df['target'].value_counts(normalize=True) * 100`, 乘以 100 是把比例转换成百分比 (如 40.95% 而不是 0.4095)。

小贴士: `value_counts()` 还有几个常用参数:

- `ascending=True`: 按频数升序排列。
- `dropna=False`: 把缺失值也统计进去 (默认会忽略 `NaN`)。
- `bins=10`: 把数值型数据分成 10 个区间统计 (适用于连续变量)。

2.3 .get(1, 0) 字典取值技巧

`target_counts` 是一个 Series, 可以像字典一样用 `.get(key, default)` 取值:

```
target_counts.get(1, 0) # 取 key=1 的值，如果不存在返回 0
```

- `target_counts.get(1, 0)` → 取 `target=1`（存活）的频数，如果数据中没有 `target=1` 的样本，返回 0。
- `target_counts.get(0, 0)` → 取 `target=0`（死亡）的频数，同理。

为什么用 `.get()` 而不是 `[]`：

- `target_counts[1]`：如果 key 不存在，会抛出 `KeyError`，程序崩溃。
- `target_counts.get(1, 0)`：如果 key 不存在，返回默认值 0，程序继续运行。

这是一种**防御性编程**的写法，让代码更健壮。虽然在本数据集中 `target=1` 和 `target=0` 都存在，但养成用 `.get()` 的习惯是好做法。

常见问题：为什么本教程要这么谨慎？因为：

1. 在某些极端情况下（比如数据加载出错），可能某一类样本不存在。
2. 这段代码可能被复用到其他数据集，其他数据集可能只有一类样本。
3. 防御性编程让代码更通用、更不容易出错。

2.4 f-string 格式化详解

本段代码用了三种 f-string 格式化语法，我们逐一拆解：

```
{target_counts.get(1, 0):>10,}
```

这是一个复合格式化，由两部分组成：`>10` 和 `,`。

- `>`：右对齐（right-align）。
- `10`：字段宽度为 10 个字符。
- `,`：千分位分隔符。

```
val = 85896
print(f"{val:>10,}") # 输出:      85,896 (前面有 4 个空格, 总共 10 字符)
print(f"{val:>,}")   # 输出: 85,896 (没有对齐)
print(f"{val:>10}")  # 输出:      85896 (没有千分位)
```

为什么右对齐：在打印表格时，数字右对齐能让小数点（或千分位）对齐，方便比较大小。比如：

```
存活 (VIVO = 1):      85,896  (40.95%)
死亡 (MORTO = 0):    123,862  (59.05%)
```

数字右对齐，看起来更整齐。

```
{target_props.get(1, 0):.2f}
```

- `.2f`：浮点数保留 2 位小数（f 表示 float）。

```
val = 40.9521
print(f"{val:.2f}") # 输出: 40.95
print(f"{val:.4f}") # 输出: 40.9521
print(f"{val:.0f}") # 输出: 41 (四舍五入到整数)
```

`{n_samples:,}`（前面已讲过）

千分位分隔符，把 `209758` 显示成 `209,758`。

小贴士：f-string 格式化语法是 `{变量:格式}`，格式部分可以组合。常用格式：

- `:`，千分位
- `:.2f` 两位小数
- `:.2%` 百分比（自动乘 100 并加 %）
- `>10` 右对齐宽 10
- `<10` 左对齐宽 10
- `^10` 居中宽 10
- `e` 科学计数法

三、平衡性判断

```
# 平衡性判断
ratio = target_props.get(1, 0) / target_props.get(0, 0)
if 0.8 <= ratio <= 1.2:
    balance_verdict = "数据基本平衡 (Balanced)"
elif 0.5 <= ratio < 0.8 or 1.2 < ratio <= 2.0:
    balance_verdict = "数据轻度不平衡 (Mildly Imbalanced)"
else:
    balance_verdict = "数据严重不平衡 (Severely Imbalanced)"

print(f"\n ▶ 平衡性判断 (Balance Assessment): {balance_verdict}")
print(f"    存活/死亡 比例 = {ratio:.3f}")
```

3.1 比例的计算

```
ratio = target_props.get(1, 0) / target_props.get(0, 0)
```

`ratio` 是存活比例 / 死亡比例，即正类比例除以负类比例。

本数据集中：

- 存活比例 = 40.95%
- 死亡比例 = 59.05%
- $ratio = 40.95 / 59.05 \approx 0.693$

3.2 三档分类的逻辑

代码用 `if-elif-else` 把数据平衡性分成三档：

条件	判定	含义
<code>0.8 <= ratio <= 1.2</code>	基本平衡	正负类比例接近 1:1
<code>0.5 <= ratio < 0.8</code> 或 <code>1.2 < ratio <= 2.0</code>	轻度不平衡	正负类比例偏离 1:1，但不极端
其他 (<code>ratio < 0.5</code> 或 <code>ratio > 2.0</code>)	严重不平衡	正负类比例严重失衡

3.3 为什么这样划分阈值

这三个阈值（0.8、1.2、0.5、2.0）是经验性的，背后逻辑是：

- **0.8–1.2（基本平衡）**：正负类比例在 4:5 到 5:4 之间，差异不大，大多数机器学习算法能直接处理，不需要特殊处理。
- **0.5–0.8 或 1.2–2.0（轻度不平衡）**：正负类比例在 1:2 到 4:5 之间，差异明显但不极端。可以尝试用 `class_weight='balanced'` 或轻微的过采样/欠采样。

- **<0.5 或 >2.0 (严重不平衡)**：正负类比例超过 1:2，差异很大。必须用 SMOTE、ADASYN 等重采样方法，或者用适合不平衡数据的评估指标（如 F1、AUC，而不是 Accuracy）。

重要概念：为什么不用 Accuracy 评估不平衡数据？

假设数据集 99% 是负类，1% 是正类。一个"全预测为负类"的傻瓜模型能达到 99% 的 Accuracy，但完全没用——它一个正类都没找出来。所以不平衡数据要用 Precision、Recall、F1、AUC 等指标。

3.4 Python 的链式比较

注意 `0.8 <= ratio <= 1.2` 这种写法，这是 Python 特有的**链式比较**，等价于：

```
0.8 <= ratio and ratio <= 1.2
```

但在 Python 中可以直接写成 `0.8 <= ratio <= 1.2`，更简洁、更接近数学表达式。这是 Python 相比其他语言的一个语法糖。

3.5 本数据集的判定结果

存活/死亡 比例 = 0.693

- `ratio = 0.693`
- 落在 `0.5 <= ratio < 0.8` 区间
- 判定为：**数据轻度不平衡 (Mildly Imbalanced)**

小贴士：本数据集的轻度不平衡在医学数据中很常见。癌症患者中死亡比例通常高于存活比例（因为癌症是严重疾病），所以 40.95% vs 59.05% 的分布在预期之内。这种程度的不平衡可以通过 `class_weight='balanced'` 处理，不需要复杂的重采样。

四、绘制标签分布图

接下来我们用 matplotlib 绘制一张包含两个子图的标签分布图：左边是条形图（计数），右边是饼图（比例）。

4.1 创建画布和子图

```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
```

`plt.subplots()` 参数详解

`plt.subplots()` 一次性创建画布（Figure）和子图（Axes），返回一个元组 `(fig, axes)`。

- `1`：子图的**行数**（1 行）。
- `2`：子图的**列数**（2 列）。
- 所以 `1, 2` 表示 1 行 2 列，共 2 个子图，水平排列。
- `figsize=(12, 5)`：画布尺寸，**单位是英寸**，格式是 `(宽, 高)`。这里宽 12 英寸、高 5 英寸。

`fig` 和 `axes` 是什么

- `fig`：Figure 对象，代表整个画布。可以理解为"一张大图"。
- `axes`：Axes 对象的数组，代表每个子图。注意"Axes"是 matplotlib 的术语，指的是"一个坐标系区域"，不是"坐标轴"。
- 当 `nrows=1` 时，`axes` 是一维数组，用 `axes[0]`、`axes[1]` 访问。

- 当 `nrows>1` 且 `ncols>1` 时, `axes` 是二维数组, 用 `axes[0][1]` 访问。

```
axes[0] # 左边的子图 (条形图)
axes[1] # 右边的子图 (饼图)
```

小贴士: `figsize` 的单位是英寸, 最终图片的像素尺寸 = `figsize × dpi`。比如 `figsize=(12, 5)` 配合 `dpi=150`, 图片就是 1800×750 像素。

4.2 准备颜色和标签

```
colors = ['#2ecc71', '#e74c3c']
labels_bar = ['VIVO\n(Alive)', 'MORTO\n(Dead)']
```

- `colors`: 两个十六进制颜色码。
- `#2ecc71`: 绿色 (代表存活, 积极)。
- `#e74c3c`: 红色 (代表死亡, 消极)。
- `labels_bar`: 条形图的 x 轴标签。
- `\n` 是换行符, 让 "VIVO" 和 "(Alive)" 分两行显示, 更紧凑。

小贴士: 颜色选择有讲究。在医学/生死场景中, 绿色=存活、红色=死亡是符合直觉的配色。如果反过来用红色表示存活, 会让读者困惑。数据可视化中, **颜色的语义要和数据的语义一致**。

4.3 绘制条形图

```
bars = axes[0].bar(labels_bar, [target_counts.get(1, 0), target_counts.get(0, 0)],
                  color=colors, edgecolor='white', width=0.5)
```

`axes[0].bar()` 参数详解

`bar()` 是绘制条形图的函数, 参数:

- 第一个参数 `labels_bar`: x 轴的标签 (类别名)。
- 第二个参数 `[target_counts.get(1, 0), target_counts.get(0, 0)]`: 每个类别对应的数值 (条形高度)。
- `color=colors`: 每个条形的填充颜色, 传入颜色列表。
- `edgecolor='white'`: 条形的**边框颜色**设为白色, 让条形之间有清晰的分隔。
- `width=0.5`: 条形的**宽度**, 默认是 0.8。设为 0.5 让条形更细, 留出更多空白, 视觉上更清爽。

返回值 `bars`

`bar()` 返回一个 `BarContainer` 对象, 包含所有条形对象。后续可以用它来在条形上添加文字标签。

4.4 在条形上添加数值标签

```
for bar, val in zip(bars, [target_counts.get(1, 0), target_counts.get(0, 0)]):
    axes[0].text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 5000,
                f'{val:,}', ha='center', va='bottom', fontsize=11, fontweight='bold')
```

这段代码的逻辑

用 `zip()` 把条形对象和对应的数值配对, 然后遍历每个条形, 在条形顶部上方添加文字。

axes[0].text() 参数详解

text(x, y, s, ...) 在坐标 (x, y) 处添加文字 s。

- **x 坐标**: `bar.get_x() + bar.get_width() / 2`
- `bar.get_x()`: 条形左边缘的 x 坐标。
- `bar.get_width()`: 条形的宽度。
- 两者相加再除以 2, 得到条形的**中心 x 坐标**。
- **y 坐标**: `bar.get_height() + 5000`
- `bar.get_height()`: 条形的高度 (即数值)。
- `+ 5000`: 在条形顶部上方 5000 单位处放文字, 避免文字和条形重叠。
- **文字内容**: `f'{val:,}'`, 数值加千分位, 如 `85,896`。
- `ha='center'`: **水平对齐** (horizontal alignment) 为居中。其他选项: `'left'`、`'right'`。
- `va='bottom'`: **垂直对齐** (vertical alignment) 为底部对齐。其他选项: `'top'`、`'center'`。
- `fontsize=11`: 字体大小。
- `fontweight='bold'`: 字体加粗。

小贴士: 在条形图上直接标注数值是好习惯, 读者不需要对照 y 轴刻度去估算。 `ha='center' + va='bottom'` 是最常见的组合: 文字水平居中、垂直方向从条形顶部向上展开。

4.5 设置标题和轴标签

```
axes[0].set_title('Target Distribution (Count)', fontsize=13, fontweight='bold')
axes[0].set_ylabel('Count')
```

- `set_title()`: 设置子图标题。
- `set_ylabel()`: 设置 y 轴标签。

4.6 隐藏多余边框 (spines)

```
axes[0].spines['top'].set_visible(False)
axes[0].spines['right'].set_visible(False)
```

什么是 spines

Spines 是 matplotlib 中“边框”的术语, 指的是子图四周的边界线。一个子图有 4 条 spines:

- `top`: 上边框
- `bottom`: 下边框 (通常保留, 因为有 x 轴)
- `left`: 左边框 (通常保留, 因为有 y 轴)
- `right`: 右边框

为什么要隐藏 top 和 right

这是数据可视化的**美学原则**:

- 顶部和右侧的边框**没有承载信息** (没有刻度、没有数据), 属于“chartjunk” (图表垃圾)。
- 隐藏它们可以让图表更简洁、更聚焦于数据本身。
- 这是 Edward Tufte 提出的“数据墨水比” (data-ink ratio) 原则的体现: 尽量减少非数据元素。

```
# 隐藏前: 四周都有边框 (封闭的方框)
# 隐藏后: 只有左和下有边框 (L 形, 更简洁)
```

小贴士：隐藏 top 和 right spines 是数据可视化的"标配"操作。seaborn 库默认就会这样做，所以用 seaborn 画的图通常更美观。如果用纯 matplotlib，建议手动隐藏这两条 spines。

五、绘制饼图

```
# 饼图
wedges, texts, autotexts = axes[1].pie(
    [target_counts.get(1, 0), target_counts.get(0, 0)],
    labels=['VIVO (Alive)', 'MORTO (Dead)'],
    autopct='%1.1f%%',
    colors=colors,
    startangle=90,
    explode=(0.03, 0.03),
    textprops={'fontsize': 11}
)
for autotext in autotexts:
    autotext.set_fontweight('bold')
axes[1].set_title('Target Distribution (Proportion)', fontsize=13, fontweight='bold')
```

5.1 axes[1].pie() 参数详解

`pie()` 是绘制饼图的函数，本教程用了多个参数：

第一个参数：数值列表

```
[target_counts.get(1, 0), target_counts.get(0, 0)]
```

每个数值决定对应扇形的大小（角度比例）。饼图会自动计算每个数值占总和的比例。

labels：每个扇形的标签

```
labels=['VIVO (Alive)', 'MORTO (Dead)']
```

这些标签会显示在每个扇形的外侧。

autopct='%1.1f%%'：自动显示百分比

`autopct` 是"automatic percentage"的缩写，控制如何在扇形上显示百分比。

- `'%1.1f%%'` 是一个格式化字符串：
- `%1.1f`：浮点数保留 1 位小数。
- `%`：转义后的百分号（单个 `%` 是格式化符号，`%%` 才表示字面意义的 `%`）。
- 所以 `40.9521%` 会显示为 `41.0%`（保留 1 位小数）。

如果设为 `autopct='%1.2f%%'`，则显示 2 位小数，如 `40.95%`。

colors=colors：扇形颜色

复用前面定义的 `colors = ['#2ecc71', '#e74c3c']`，绿色和红色。

startangle=90：起始角度

`startangle` 控制第一个扇形从哪个角度开始绘制。

- `startangle=90`：从**正上方**（12 点钟方向）开始，逆时针绘制。
- `startangle=0`（默认）：从正右方（3 点钟方向）开始。

- `startangle=180`：从正下方（6 点钟方向）开始。

设为 90 是惯例，让饼图从顶部开始，更符合阅读习惯。

`explode=(0.03, 0.03)`：扇形分离

`explode` 是一个元组，控制每个扇形**向外偏移**的距离。

- `(0.03, 0.03)`：两个扇形都向外偏移 0.03（半径的 3%）。
- `(0.1, 0)`：第一个扇形偏移 0.1，第二个不偏移。
- `(0, 0)`：不偏移（默认）。

本教程让两个扇形都轻微偏移（0.03），制造一种"分离"的视觉效果，让饼图更有层次感。

小贴士： `explode` 常用于**强调某个类别**。比如你想突出"死亡"类，可以设 `explode=(0, 0.1)`，让死亡扇形向外突出。本教程两个都偏移是为了对称美观。

`textprops={'fontsize': 11}`：文字属性

`textprops` 是一个字典，设置所有文字（标签和百分比）的属性。

- `{'fontsize': 11}`：字体大小设为 11。
- 还可以设置 `{'fontsize': 11, 'color': 'white'}` 等。

5.2 返回值 `wedges, texts, autotexts`

`pie()` 返回三个对象：

- `wedges`：扇形对象列表（Wedge 对象）。
- `texts`：外侧标签文字对象列表（"VIVO (Alive)"、"MORTO (Dead)"）。
- `autotexts`：扇形内部百分比文字对象列表（"41.0%"、"59.0%"）。

5.3 加粗百分比文字

```
for autotext in autotexts:
    autotext.set_fontweight('bold')
```

遍历所有百分比文字，调用 `set_fontweight('bold')` 加粗。这样百分比更醒目，而外侧标签保持普通字重，形成视觉层次。

5.4 设置饼图标题

```
axes[1].set_title('Target Distribution (Proportion)', fontsize=13, fontweight='bold')
```

和条形图标题风格保持一致。

六、保存和关闭图形

```
plt.tight_layout()
plt.savefig(os.path.join(IMG_DIR, "01_target_distribution.png"), dpi=150, bbox_inches='tight')
plt.close()
print(" [图] 01_target_distribution.png → 标签分布图已保存")
```

6.1 plt.tight_layout() 的作用

`tight_layout()` 会**自动调整子图间距和边距**，避免子图之间、子图与标题之间、子图与坐标轴标签之间重叠。

- 不调用时：matplotlib 用默认边距，可能导致标题和子图重叠、子图之间间距过大或过小。
- 调用后：自动计算最优布局，让图表更紧凑、更美观。

小贴士：只要画了多个子图，就应该调用 `tight_layout()`，这是几乎必须的步骤。它能让你的图表瞬间专业很多。

6.2 plt.savefig() 参数详解

`savefig()` 把当前图形保存到文件。

文件路径

```
os.path.join(IMG_DIR, "01_target_distribution.png")
```

用 `os.path.join()` 拼接路径，保存到 `img/01_target_distribution.png`。文件扩展名 `.png` 决定了输出格式，也可以用 `.jpg`、`.pdf`、`.svg` 等。

dpi=150：分辨率

`dpi` 是 **dots per inch**（每英寸点数），控制输出图片的分辨率。

- `dpi=72`：屏幕显示常用，文件小。
- `dpi=150`：本教程使用，适合屏幕显示和一般打印，文件大小适中。
- `dpi=300`：印刷质量，文件较大。

图片的像素尺寸 = `figsize × dpi`。本教程 `figsize=(12, 5)`，`dpi=150`，所以图片是 1800×750 像素。

常见问题：dpi 设多少合适？

- 论文/报告插图：300 dpi（印刷质量）
- 网页/PPT 展示：150 dpi（足够清晰，文件不太大）
- 快速预览：72 dpi（最小）

注意 `dpi` 越高，文件越大。本教程选 150 是平衡清晰度和文件大小的选择。

bbox_inches='tight'：紧凑裁剪

`bbox_inches='tight'` 让保存时**自动裁剪掉图形周围的空白**，只保留有内容的区域。

- 不设置时：保存的图片四周会有较大空白，尤其是有标题、标签溢出时。
- 设置 `'tight'`：图片紧贴内容，没有多余白边。

这和 `tight_layout()` 是配套使用的：`tight_layout()` 调整内部布局，`bbox_inches='tight'` 裁剪外部空白。

6.3 plt.close() 为什么要关闭图形

```
plt.close()
```

`plt.close()` 关闭当前图形，释放内存。

为什么要关闭：

1. **释放内存**：matplotlib 的 Figure 对象会保留所有绘图数据，如果不关闭，内存会持续累积。在循环中画很多图时尤其重要。
2. **避免显示冲突**：在 Jupyter Notebook 中，不关闭的图形可能会被重复显示。
3. **良好习惯**：画完图就关闭，是 matplotlib 的最佳实践。

常见问题：在循环中画图时，如果不调用 `plt.close()`，内存会暴涨甚至 OOM。正确写法：

```
python for i in range(100): plt.figure() plt.plot(...) plt.savefig(f"fig_{i}.png") plt.close()
# 必须关闭！
```

七、运行结果

7.1 控制台输出

运行本模块代码后，控制台输出大致如下：

```
=====
内容 1：数据概况
=====

▶ 样本数量 (Number of samples): 209,758
▶ 特征数量 (Number of features): 39
▶ 其中包含 38 个原始特征 + 1 个目标变量 (target)

▶ 标签分布 (Target Distribution):
  存活 (VIVO = 1):      85,896   (40.95%)
  死亡 (MORTO = 0):    123,862   (59.05%)

▶ 平衡性判断 (Balance Assessment): 数据轻度不平衡 (Mildly Imbalanced)
  存活/死亡 比例 = 0.693
[图] 01_target_distribution.png → 标签分布图已保存
```

7.2 关键数字汇总

指标	数值
样本数量	209,758
特征数量	39 (38 原始 + 1 target)
存活 (VIVO=1)	85,896 (40.95%)
死亡 (MORTO=0)	123,862 (59.05%)
存活/死亡比例	0.693
平衡性判定	轻度不平衡 (Mildly Imbalanced)

7.3 标签分布图

标签分布图

图表解读：

- **左图（条形图）：**展示每个类别的绝对数量。绿色条形（VIVO）高度为 85,896，红色条形（MORTO）高度为 123,862。可以直观看到死亡样本比存活样本多约 38,000。
- **右图（饼图）：**展示每个类别的比例。绿色扇形占 41.0%，红色扇形占 59.0%。可以直观看到死亡占多数，但差距不算极端。

小贴士：条形图和饼图各有优势：

- **条形图：**适合比较绝对数量，差异更直观。
- **饼图：**适合展示比例关系，强调“部分占整体”。

八、本模块小结

核心知识点回顾

1. **数据形状**: `df.shape` 返回 (行数, 列数), `df.shape[0]` 是行数, `df.shape[1]` 是列数。
2. **标签分布统计**:
 - `value_counts()` 统计频数。
 - `value_counts(normalize=True)` 统计比例。
 - `.get(key, default)` 防御性取值, 避免 `KeyError`。
3. **f-string 格式化**:
 - `{:,}` 千分位
 - `{:.2f}` 两位小数
 - `{:>10,}` 右对齐 + 千分位
4. **平衡性判断**:
 - ratio = 正类比例 / 负类比例
 - 0.8-1.2: 基本平衡
 - 0.5-0.8 或 1.2-2.0: 轻度不平衡
 - 其他: 严重不平衡
5. **matplotlib 绘图**:
 - `plt.subplots(1, 2, figsize=(12, 5))` 创建 1 行 2 列子图。
 - `bar()` 画条形图, `pie()` 画饼图。
 - `text()` 在图上添加文字标签。
 - 隐藏 `top` 和 `right` spines 提升美观度。
6. **图形保存**:
 - `tight_layout()` 调整内部布局。
 - `savefig(dpi=150, bbox_inches='tight')` 保存高分辨率紧凑图片。
 - `plt.close()` 释放内存。

本数据集的关键发现

- 数据集有 209,758 条记录, 39 列 (38 原始特征 + 1 target)。
- 标签分布: 存活 40.95%, 死亡 59.05%。
- 数据**轻度不平衡**, 比例 0.693。
- 这种程度的不平衡在医学数据中常见, 后续建模时可以用 `class_weight='balanced'` 处理。

下一模块预告

本模块完成了数据概况分析, 知道了数据集的大小和标签分布。下一模块 (**模块 2: 缺失值分析**) 将深入分析:

- 每个特征的缺失比例
- 缺失模式 (MCAR/MAR/MNAR)
- 缺失值之间的相关性
- 缺失值的可视化

思考题:

1. 如果数据集严重不平衡 (比如 1:99), 你会如何调整本模块的可视化代码?
2. 为什么 `ratio = target_props.get(1, 0) / target_props.get(0, 0)` 用比例相除, 而不是用频数相除? 两者

结果一样吗？

3. 饼图在什么情况下不适合使用？条形图相比饼图有什么优势？

4. 如果要把这张图保存成 PDF 格式用于论文，需要修改哪些参数？

模块 2：缺失值分析

学习目标

完成本模块后，你将能够：

1. **理解缺失值的概念**：知道什么是缺失值、为什么真实数据中会普遍存在缺失。
2. **掌握 Pandas 缺失值检测方法**：熟练使用 `df.isnull()`、`.sum()`、`.astype()` 等方法统计缺失情况。
3. **构建缺失值汇总表**：能用 `pd.DataFrame({...})` 把 Series 整理成可读的表格，并用 `sort_values` 排序。
4. **编写分档函数**：会用 `def` 定义函数、用 `if-elif-else` 实现分档逻辑，并通过 `.apply()` 应用到 Series。
5. **绘制水平条形图**：理解 `ax.barh()` 与 `ax.bar()` 的区别，掌握 `invert_yaxis()`、`ax.text()` 标签定位技巧。
6. **绘制缺失模式热力图**：理解 `sns.heatmap()` 的核心参数（`cmap`、`cbar_kws`、`linewidths`），知道为什么要抽样。
7. **绘制缺失相关性热力图**：理解 `missing_matrix.corr()` 的含义，掌握上三角掩码 `np.triu()` 的原理与用法。
8. **区分三种缺失机制**：能区分 MCAR / MAR / MNAR，并能结合本数据集给出合理示例。
9. **解读分析结果**：能从图中读出“哪些列缺失最严重”、“哪些列的缺失模式高度相关”等关键信息。

前置知识：Python 基础语法、Pandas 的 DataFrame/Series 概念、Matplotlib 的 `subplots` 基本用法、NumPy 数组基础操作。

一、为什么必须做缺失值分析？

真实世界的医学数据几乎从不“干净”。在巴西癌症登记数据集中，我们已经过滤掉 `Status.Vital` 缺失的样本，剩余 **209,758 条记录、39 列**（38 个原始特征 + 1 个目标变量）。但即便如此，许多特征列仍然存在大量缺失。

缺失值分析要回答三个核心问题：

1. **缺失有多严重？** —— 哪些列缺失多到几乎不可用？
2. **缺失有没有规律？** —— 缺失是随机发生的，还是和某些变量有关？
3. **缺失的机制是什么？** —— 是 MCAR、MAR 还是 MNAR？这决定了我们能否直接删除、能否用均值填补。

重要概念：缺失值分析不是“清理数据”的附属步骤，而是理解数据生成过程（Data Generating Process）的关键窗口。**缺失本身往往携带着信息**——例如“病情严重的患者更可能缺失教育信息”，这种规律本身就是建模时可以利用的信号。

二、计算各列缺失比例

2.1 代码

```
# 计算各列缺失比例
missing_series = df.isnull().sum()
missing_pct = (missing_series / len(df)) * 100
```

2.2 逐行解析

df.isnull() —— 返回一个布尔 DataFrame

df.isnull() (等价于 **df.isna()**) 会对 DataFrame 的**每一个单元格**进行判断：如果该单元格是 **NaN**、**None**、**NaT** 等"缺失值", 就填 **True**；否则填 **False**。

关键理解：**df.isnull()** 返回的不是一个数字表，而是一个**与原 DataFrame 形状完全相同的布尔 DataFrame**（每个元素都是 **True** 或 **False**）。

举例：假设原数据是

Name	Age	City
Alice	25	NY
Bob	NaN	LA
NaN	30	NaN

那么 **df.isnull()** 返回：

Name	Age	City
False	False	False
False	True	False
True	False	True

.sum() —— 默认按列求和

在 Pandas 中，**DataFrame.sum()** 默认参数是 **axis=0**，也就是**按列求和**（把每一列的所有行加起来）。

由于布尔值在求和时会被当作整数（**True = 1**，**False = 0**），所以 **df.isnull().sum()** 的结果就是：**每一列有多少个缺失值**。

返回结果是一个 **Series**，索引是列名，值是该列的缺失计数：

```
Age      1
City     1
Name     1
dtype: int64
```

小贴士：如果你想按行统计每个样本缺失了多少个字段，用 **df.isnull().sum(axis=1)**。这在检测"几乎全空"的样本时很有用。

```
missing_pct = (missing_series / len(df)) * 100
```

- **len(df)** 返回 DataFrame 的行数（这里是 209,758）。
- **missing_series / len(df)** 是 Pandas 的**广播运算**：Series 的每个元素都除以同一个数。

- `* 100` 把比例转成百分比。

最终 `missing_pct` 也是一个 Series，索引与 `missing_series` 相同，值是每列的缺失百分比。

三、构建缺失值汇总表

3.1 代码

```
missing_df = pd.DataFrame({
    'Column': missing_series.index,
    'Missing_Count': missing_series.values,
    'Missing_Pct': missing_pct.values
}).sort_values('Missing_Pct', ascending=False)
```

3.2 逐行解析

`pd.DataFrame({...})` —— 用字典构造 DataFrame

`pd.DataFrame()` 有多种构造方式，最常用的一种是**传入一个字典**：字典的 key 成为列名，value（通常是列表或数组）成为该列的数据。

这里我们传入三个 key-value 对：

字典 key	字典 value	含义
'Column'	<code>missing_series.index</code>	列名（Series 的索引）
'Missing_Count'	<code>missing_series.values</code>	缺失计数（Series 的值数组）
'Missing_Pct'	<code>missing_pct.values</code>	缺失百分比

为什么用 `.values`？ Series 的 `.values` 属性返回底层的 NumPy 数组（去掉索引）。如果我们直接传 Series，Pandas 会尝试按索引对齐，虽然结果通常一样，但显式用 `.values` 更清晰、避免对齐歧义。

`.sort_values('Missing_Pct', ascending=False)` —— 按缺失比例降序排列

`sort_values()` 是 DataFrame 的排序方法，关键参数：

参数	说明
<code>by</code>	排序依据的列名（或列名列表）。这里是 <code>'Missing_Pct'</code> 。
<code>ascending</code>	是否升序。 <code>True</code> （默认）从小到大； <code>False</code> 从大到小。我们想要缺失最多的在最上面，所以用 <code>False</code> 。
<code>inplace</code>	是否原地修改。默认 <code>False</code> ，返回新 DataFrame。
<code>na_position</code>	缺失值放在开头还是末尾，默认 <code>'last'</code> 。

排序后，`missing_df` 的第一行就是缺失比例最高的列。

四、过滤出"有缺失"的列

4.1 代码

```
# 只显示缺失 > 0 的列
missing_nonzero = missing_df[missing_df['Missing_Pct'] > 0]
```

4.2 原理：布尔索引 (Boolean Indexing)

这是 Pandas 最常用的筛选技巧，分两步理解：

第一步： `missing_df['Missing_Pct'] > 0` 会返回一个布尔 Series（与 `missing_df` 行数相同），每一行是 `True` 或 `False`，表示该列的缺失比例是否大于 0。

```
0      True
1      True
2      True
...
36     False    # 这一列没有缺失
37     False
Name: Missing_Pct, dtype: bool
```

第二步： `missing_df[布尔Series]` 会只保留布尔值为 `True` 的行，丢弃 `False` 的行。这就像用一个“掩码”盖在 DataFrame 上，只露出我们想要的部分。

小贴士： 布尔索引可以组合多个条件，用 `&`（与）、`|`（或）、`~`（非）连接，每个条件必须用括号括起来。例如：

```
```python
```

### 缺失比例在 5% 到 50% 之间的列

```
missing_df[(missing_df['Missing_Pct'] >= 5) & (missing_df['Missing_Pct'] < 50)]
```
```

五、缺失值分档统计

5.1 代码

```
# 缺失值分档统计
def missing_category(pct):
    if pct == 0:
        return "无缺失"
    elif pct < 5:
        return "轻度 (< 5%)"
    elif pct < 20:
        return "中度 (5%-20%)"
    elif pct < 50:
        return "高度 (20%-50%)"
    else:
        return "严重 (≥ 50%)"

missing_df['Category'] = missing_df['Missing_Pct'].apply(missing_category)
```

5.2 逐行解析

`def missing_category(pct):` —— 函数定义

`def` 是 Python 定义函数的关键字，语法是：

```
def 函数名(参数1, 参数2, ...):
    函数体
    return 返回值
```

这里定义了一个名为 `missing_category` 的函数，接收一个参数 `pct`（百分比），返回一个字符串（分档标签）。

`if-elif-else` —— 多分支条件判断

Python 的条件判断结构：

- `if 条件1:` —— 如果条件1为真，执行这块代码。
- `elif 条件2:` —— 否则如果条件2为真，执行这块代码（`elif` = `else if`）。
- `else:` —— 以上条件都不满足时执行。

关键理解：`if-elif-else` 是**从上到下依次判断，一旦某个条件为真就执行对应代码块，然后跳出整个结构**，不会再检查后面的条件。所以条件的顺序很重要。

本函数的分档逻辑：

| 输入 <code>pct</code> | 命中分支 | 返回值 |
|--------------------------------|-----------------------|----------------|
| <code>pct == 0</code> | 第一个 <code>if</code> | "无缺失" |
| <code>0 < pct < 5</code> | 第一个 <code>elif</code> | "轻度 (< 5%)" |
| <code>5 ≤ pct < 20</code> | 第二个 <code>elif</code> | "中度 (5%-20%)" |
| <code>20 ≤ pct < 50</code> | 第三个 <code>elif</code> | "高度 (20%-50%)" |
| <code>pct ≥ 50</code> | <code>else</code> | "严重 (≥ 50%)" |

常见问题：为什么用 `elif` 而不是多个独立的 `if`？

如果写成多个独立的 `if`，每个条件都会被独立判断。例如 `pct = 3` 时，`pct < 5` 为真，`pct < 20` 也为真，会导致逻辑混乱。`elif` 保证了"互斥"——只有一个分支会被执行。

`.apply(missing_category)` —— 把函数应用到 Series 的每个元素

`Series.apply(func)` 是 Pandas 的核心方法之一，它的作用是：**对 Series 中的每一个元素，调用 `func` 函数，并把所有返回值组成一个新的 Series。**

可以理解为等价的循环：

```
# apply 的等价写法（伪代码）
result = []
for value in missing_df['Missing_Pct']:
    result.append(missing_category(value))
missing_df['Category'] = result
```

但 `apply` 用的是底层 C 实现，比显式循环快得多，而且代码更简洁。

小贴士：`apply` 也可以用在 DataFrame 上（`df.apply(func, axis=0/1)`），`axis=0` 对每列应用函数，`axis=1` 对每行应用函数。

六、图 2a：缺失比例条形图（Top 30）

6.1 代码

```
# 图 2a：缺失比例条形图（Top 30）
plot_missing = missing_nonzero.head(30)
fig, ax = plt.subplots(figsize=(12, 8))
bars = ax.barh(range(len(plot_missing)), plot_missing['Missing_Pct'].values,
               color='#3498db', edgecolor='white')
ax.set_yticks(range(len(plot_missing)))
ax.set_yticklabels(plot_missing['Column'].values, fontsize=9)
ax.set_xlabel('Missing Percentage (%)', fontsize=12)
ax.set_title('Top 30 Features by Missing Value Percentage', fontsize=14, fontweight='bold')
ax.invert_yaxis()
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)

# 添加数值标签
for bar, pct in zip(bars, plot_missing['Missing_Pct'].values):
    ax.text(bar.get_width() + 0.3, bar.get_y() + bar.get_height() / 2,
            f'{pct:.1f}%', va='center', fontsize=8)
```

6.2 逐行解析

ax.barh() vs **ax.bar()** —— 水平条形图 vs 垂直条形图

| 方法 | 方向 | x 轴含义 | y 轴含义 | 适用场景 |
|------------------|----|------------|------------|------------------------------|
| ax.bar() | 垂直 | 类别（柱子横向排列） | 数值（柱子高度） | 类别较少、类别名短 |
| ax.barh() | 水平 | 数值（柱子长度） | 类别（柱子纵向排列） | 类别多、类别名长 （本例 30 个长列名） |

为什么本例用 barh？ 因为我们要展示 30 个特征，且列名（如 `Code.of.Disease.Adult.Young.`）很长。如果用垂直柱状图，x 轴标签会挤在一起或需要旋转 90°，可读性很差。水平条形图让类别名在 y 轴水平排列，天然适合这种场景。

barh 的参数：

| 参数 | 说明 |
|---------------------------|---|
| 第 1 个位置参数（ y ） | 柱子的 y 坐标位置，这里是 <code>range(30)</code> ，即 0,1,2,...,29 |
| 第 2 个位置参数（ width ） | 柱子的长度（数值），这里是缺失百分比 |
| color | 柱子填充颜色， <code>'#3498db'</code> 是十六进制颜色码（蓝色） |
| edgecolor | 柱子边框颜色， <code>'white'</code> 让柱子之间有清晰分隔 |

返回值 **bars** 是一个 **BarContainer** 对象，包含所有柱子的引用，后面添加标签时要用到。

ax.invert_yaxis() —— 反转 y 轴

默认情况下，Matplotlib 的 y 轴是**从下到上递增**的（0 在底部，29 在顶部）。这意味着 **barh** 画的第一个柱子（y=0）会在图的**底部**。

但我们习惯从上往下读，希望缺失最多的列（排序后第一行）显示在**顶部**。**ax.invert_yaxis()** 把 y 轴反转，让 0 在顶部、29 在底部，这样第一个条目就显示在最上面。

常见问题：如果不加 **invert_yaxis()**，图会“倒过来”——缺失最多的列在底部，看起来很别扭。这是画水平条形图时几乎必加的一行。

`ax.spines['top'].set_visible(False)` —— 隐藏顶部和右侧边框

Matplotlib 默认会给予图加一个"方框"（四条边：top、bottom、left、right）。现代数据可视化倾向于"简洁"风格，通常会隐藏 top 和 right 两条边，只保留 left 和 bottom 形成一个"L"形坐标轴。

```
ax.spines['top'].set_visible(False)    # 隐藏顶部边框
ax.spines['right'].set_visible(False)  # 隐藏右侧边框
```

`ax.text()` —— 在条形上添加数值标签

这是最需要理解的细节之一。`ax.text(x, y, s)` 在坐标 `(x, y)` 处绘制字符串 `s`。

在循环中：

```
for bar, pct in zip(bars, plot_missing['Missing_Pct'].values):
    ax.text(bar.get_width() + 0.3, bar.get_y() + bar.get_height() / 2,
            f'{pct:.1f}%', va='center', fontsize=8)
```

- `zip(bars, pct_values)` 把柱子对象和对应的百分比值"配对"成元组，方便同时遍历。
- `bar.get_width()` —— 返回柱子的长度（即缺失百分比数值）。
- `bar.get_y()` —— 返回柱子左下角的 y 坐标。
- `bar.get_height()` —— 返回柱子的"高度"（在水平条形图中，这是柱子的粗细程度）。

坐标计算逻辑：

| 坐标 | 计算 | 含义 |
|----|---|----------------------------|
| x | <code>bar.get_width() + 0.3</code> | 柱子末端再往右偏移 0.3 个单位，让标签不贴着柱子 |
| y | <code>bar.get_y() + bar.get_height() / 2</code> | 柱子的垂直中心（柱子起点 + 一半高度） |
| s | <code>f'{pct:.1f}%'</code> | 格式化字符串，保留 1 位小数 |

`va='center'` 是 vertical alignment（垂直对齐）参数，让文字的垂直中心对齐到指定的 y 坐标。如果不加，文字会默认底部对齐，看起来偏上。

小贴士： `f'{pct:.1f}%'` 是 f-string 格式化语法，`:.1f` 表示"浮点数，保留 1 位小数"。例如 `pct = 77.15` 会渲染成 `"77.2%"`。

6.3 结果图

缺失比例图

从图中可以清晰看到：

- `Child.Illness.Code` 和 `Child.Illness.Description` 缺失率最高，达到 **98.69%**——这两个字段是"儿童疾病"相关，绝大多数成年癌症患者自然没有这些信息，所以几乎全空。
- `TNM` 缺失 **77.15%**、`Extension` 缺失 **60.58%**——这两个是肿瘤临床分期关键字段，缺失率偏高，说明很多病例的分期记录不完整。
- `Age`、`Date.of.Birth` 缺失率很低（< 0.3%），属于基础人口学信息，记录比较完整。

七、图 2b：缺失模式热力图

7.1 代码

```
# 图 2b：缺失矩阵热力图（选取关键列）
key_cols_candidates = ['Age', 'Gender', 'Raca.Color', 'Degree.of.Education',
                       'State.Civil', 'TNM', 'Extension', 'Laterality',
                       'Diagnostic.means', 'Type.of.Death', 'Distant.metastasis']
key_cols = [c for c in key_cols_candidates if c in df.columns]
missing_matrix = df[key_cols].isnull().astype(int)

# 如果数据太大，抽样显示
sample_size = min(5000, len(df))
if len(df) > sample_size:
    np.random.seed(42)
    sample_idx = np.random.choice(len(df), sample_size, replace=False)
    missing_matrix_sample = missing_matrix.iloc[sample_idx]
else:
    missing_matrix_sample = missing_matrix

fig, ax = plt.subplots(figsize=(14, 8))
sns.heatmap(missing_matrix_sample.T, cmap=['#2ecc71', '#e74c3c'],
            cbar_kws={'label': 'Missing (1) / Present (0)', 'ticks': [0, 1]},
            ax=ax, linewidths=0)
```

7.2 逐行解析

列表推导式：`key_cols = [c for c in key_cols_candidates if c in df.columns]`

这是一种 Pythonic 的写法，等价于：

```
key_cols = []
for c in key_cols_candidates:
    if c in df.columns:
        key_cols.append(c)
```

作用是：从候选列名列表中，**只保留实际存在于 DataFrame 中的列**，避免因列名拼写错误或数据版本不同而报错。

`df[key_cols].isnull().astype(int)` —— 为什么要把布尔转成 int？

分两步：

1. `df[key_cols].isnull()` 返回一个布尔 DataFrame（True / False）。
2. `.astype(int)` 把布尔值转换成整数：True → 1，False → 0。

为什么要转换？ 因为 `sns.heatmap()` 需要的是**数值矩阵**——它根据每个单元格的数值映射到颜色。如果传入布尔值，虽然 Seaborn 内部可能也能处理，但显式转成 `int` 更规范、更可控，也方便后续计算相关性（`corr()` 要求数值类型）。

转换后，`1` 表示“缺失”，`0` 表示“存在”，这样热力图就能用颜色直观区分。

`np.random.seed(42)` 和 `np.random.choice()` —— 随机抽样

```
sample_size = min(5000, len(df))
if len(df) > sample_size:
    np.random.seed(42)
    sample_idx = np.random.choice(len(df), sample_size, replace=False)
    missing_matrix_sample = missing_matrix.iloc[sample_idx]
```

为什么要抽样？ 我们有 209,758 行数据。如果全部画在热力图上，每个像素代表一行，图会非常巨大且密集，根本看不出模式。抽样 5000 行既能保留整体的缺失模式，又能让图清晰可读。

`np.random.seed(42)` —— 设置随机数种子。计算机的"随机数"其实是伪随机（由算法生成），给定相同的种子，每次运行生成的随机序列都**完全相同**。这样：

- 结果**可复现**：别人运行你的代码能得到一样的图。
- 调试方便：如果图有问题，每次都能复现同样的样本。

为什么是 42？ 这是一个程序员圈的"梗"——来自《银河系漫游指南》中"生命、宇宙及一切的终极答案是 42"。用 42 只是约定俗成，用 0、1、123 都可以，关键是**固定一个值**。

`np.random.choice(a, size, replace)` —— 从数组中随机抽样：

| 参数 | 说明 |
|----------------------|---|
| <code>a</code> | 抽样的总体。这里是 <code>len(df)</code> ，等价于从 <code>0, 1, 2, ..., 209757</code> 中抽 |
| <code>size</code> | 抽多少个。这里是 <code>sample_size = 5000</code> |
| <code>replace</code> | 是否有放回。 <code>False</code> 表示 无放回抽样 （同一个样本不会被抽两次） |

返回的是 5000 个随机索引，然后用 `missing_matrix.iloc[sample_idx]` 按这些索引取行，得到抽样后的子矩阵。

`sns.heatmap()` —— **热力图核心函数**

```
sns.heatmap(missing_matrix_sample.T, cmap=['#2ecc71', '#e74c3c'],
            cbar_kws={'label': 'Missing (1) / Present (0)', 'ticks': [0, 1]},
            ax=ax, linewidths=0)
```

.T —— 转置矩阵。原矩阵是"行=样本，列=特征"，转置后变成"行=特征，列=样本"，这样特征名显示在 y 轴（垂直排列，更易读），样本在 x 轴。

关键参数详解：

| 参数 | 说明 |
|-------------------------|--|
| <code>data</code> | 第一个位置参数，要绘制的二维数值矩阵 |
| <code>cmap</code> | 颜色映射。 传列表的含义 ：当数据只有 0 和 1 两个离散值时，传 <code>['#2ecc71', '#e74c3c']</code> （绿色、红色）表示 0 用第一种颜色（绿=存在），1 用第二种颜色（红=缺失）。这种"红绿配色"非常直观——红色一眼就能看出缺失区域 |
| <code>cbar_kws</code> | colorbar kwargs 的缩写，是一个字典，用来配置右侧的颜色条。 <code>'label'</code> 设置颜色条的标题， <code>'ticks'</code> 设置颜色条上显示的刻度位置（这里只显示 0 和 1 两个刻度） |
| <code>ax</code> | 在哪个子图上绘制 |
| <code>linewidths</code> | 单元格之间的间隔线宽度。 <code>0</code> 表示无间隔，让色块紧密相连，适合大规模矩阵 |

小贴士： `cmap` 传列表的方式只适合离散值（如 0/1）。如果是连续值，应该传 `colormap` 名称（如 `'Blues'`、`'viridis'`）。

7.3 结果图

缺失模式热力图

如何解读这张图？

- **每一行**是一个特征，**每一列**是一个样本（共 5000 个抽样样本）。
- **绿色 = 数据存在 (0)**，**红色 = 数据缺失 (1)**。

- `Age`、`Gender` 几乎全绿 → 记录非常完整。
- `Distant.metastasis`、`TNM`、`Extension`、`Type.of.Death` 有大片红色 → 缺失严重。
- 观察"红色条纹是否对齐": 如果多个特征的红色出现在**同一批样本**上, 说明这些特征的缺失是**相关的** (可能是同一类患者整体记录不全)。

八、图 2c: 缺失值相关性热力图

8.1 代码

```
# 图 2c: 缺失值相关性热力图
missing_corr = missing_matrix.corr()
fig, ax = plt.subplots(figsize=(10, 8))
mask = np.triu(np.ones_like(missing_corr, dtype=bool), k=1)
sns.heatmap(missing_corr, mask=mask, cmap='RdBu_r', center=0,
            annot=True, fmt='.2f', square=True,
            linewidths=0.5, cbar_kws={'label': 'Correlation'},
            ax=ax, annot_kws={'fontsize': 8})
```

8.1 逐行解析

`missing_matrix.corr()` —— 计算的是什么相关性?

`missing_matrix` 是一个 0/1 矩阵 (1=缺失, 0=存在), `corr()` 计算**列与列之间的皮尔逊相关系数**。

关键理解: 这里计算的**不是**"原始特征值"之间的相关性, 而是**"缺失模式"之间的相关性**——即"特征 A 缺失"这件事, 与"特征 B 缺失"这件事, 是否倾向于同时发生。

- 相关系数 **接近 +1**: 两个特征倾向于**同时缺失** (同生共死)。
- 相关系数 **接近 -1**: 一个特征缺失时, 另一个倾向于**不缺失** (此消彼长)。
- 相关系数 **接近 0**: 两个特征的缺失**互不影响**。

重要概念: 缺失相关性是判断缺失机制的重要线索。如果多个临床字段 (如 `TNM`、`Extension`、`Laterality`) 的缺失高度正相关, 往往说明这些字段来自同一套检查流程——没做这套检查的患者, 这些字段会一起缺失。

`np.triu(np.ones_like(missing_corr, dtype=bool), k=1)` —— 生成上三角掩码

这是 NumPy 的一个经典技巧, 分两步理解:

第一步: `np.ones_like(missing_corr, dtype=bool)`

生成一个与 `missing_corr` 形状相同的全 `True` 矩阵 (因为 `dtype=bool`, 所以 `1` 变成 `True`)。假设 `missing_corr` 是 11×11 , 这就是一个 11×11 的全 `True` 矩阵。

第二步: `np.triu(..., k=1)`

`triu` 是 "triangle upper" 的缩写, 返回矩阵的**上三角部分**:

- `k=0`: 保留主对角线及以上的部分 (主对角线 + 上三角)。
- `k=1`: 保留**主对角线以上**的部分 (**不包括主对角线**), 主对角线及以下全变 `False`。
- `k=2`: 从主对角线再往上偏移一行。

举例 (4×4 矩阵, `k=1`):

```

原矩阵 (全 True)          triu(k=1) 后
T T T T                  F T T T
T T T T      →          F F T T
T T T T                  F F F T
T T T T                  F F F F

```

为什么要生成这个掩码？ 因为相关性矩阵是**对称的**（`corr(A,B) == corr(B,A)`），而且主对角线永远是 1（`corr(A,A) == 1`，没意义）。如果全画出来，一半的信息是冗余的。用上三角掩码可以**只保留下三角（含或不含对角线）**，让图更简洁。

mask=mask —— 如何隐藏上三角？

`sns.heatmap()` 的 `mask` 参数接收一个**布尔矩阵**，规则是：**True** 的位置会被隐藏（不绘制）。

我们生成的 `mask` 中，上三角是 `True`，所以上三角被隐藏；下三角是 `False`，所以下三角正常显示。最终图上只显示下三角部分。

sns.heatmap() 的其他参数详解

```

sns.heatmap(missing_corr, mask=mask, cmap='RdBu_r', center=0,
             annot=True, fmt='.2f', square=True,
             linewidths=0.5, cbar_kws={'label': 'Correlation'},
             ax=ax, annot_kws={'fontsize': 8})

```

| 参数 | 说明 |
|-------------------------|--|
| <code>cmap</code> | 'RdBu_r' = Red-Blue-reversed。RdBu 是"红-白-蓝"渐变，加 <code>_r</code> 表示反转，变成"蓝-白-红"。这样 负相关=蓝色，0=白色，正相关=红色 ，非常直观 |
| <code>center</code> | 颜色映射的中心值。 <code>center=0</code> 让 0 对应白色（中间色），负值偏蓝、正值偏红。如果不设 <code>center</code> ，颜色会以数据的中位数为中心，可能导致 0 不是白色，误导解读 |
| <code>annot</code> | annotation 的缩写， <code>True</code> 表示在每个格子里 标注数值 |
| <code>fmt</code> | format string，'.2f' 表示保留 2 位小数。只有 <code>annot=True</code> 时才生效 |
| <code>square</code> | <code>True</code> 让每个单元格是正方形（而不是根据数据范围自动拉伸）。相关性矩阵通常用正方形更美观 |
| <code>linewidths</code> | 单元格之间的间隔线宽度， <code>0.5</code> 让格子之间有细线分隔，更易读 |
| <code>cbar_kws</code> | 颜色条配置，'label' 设置颜色条标题 |
| <code>annot_kws</code> | annotation kwargs，标注文字的样式。{'fontsize': 8} 设置字号为 8，避免格子装不下 |

小贴士： `cmap='RdBu_r' + center=0` 是绘制相关性矩阵的"黄金组合"——红色代表正相关、蓝色代表负相关、白色代表无相关，一目了然。记住这个搭配，几乎所有相关性热力图都适用。

8.3 结果图

缺失相关性热力图

如何解读这张图？

- 颜色越**红**，两个特征的缺失越倾向于**同时发生**。
- 颜色越**蓝**，一个缺失时另一个越倾向于**不缺失**。
- 颜色越**白**，两个特征的缺失**互不相关**。

典型观察：`TNM`、`Extension`、`Laterality` 这些临床分期相关字段，往往呈现较强的正相关——它们来自同一套肿瘤分期评估流程，没做评估的患者这些字段会一起缺失。

九、实际运行结果

9.1 缺失概况

存在缺失值的列数：24 / 39
完全完整的列数：15 / 39

关键数字：24/39 列存在缺失（约 62% 的列不完整），其中 **10 列缺失 $\geq 50\%$** 。这说明本数据集的缺失问题相当严重，建模时必须谨慎处理。

9.2 各列缺失比例（按严重程度分档）

| 程度 | 列数 | 代表列（缺失率） |
|--------------------|----|--|
| 严重 ($\geq 50\%$) | 10 | <code>Child.Illness.Code</code> (98.69%)、 <code>TNM</code> (77.15%)、 <code>Extension</code> (60.58%) |
| 高度 (20%-50%) | 5 | <code>Laterality</code> (49.65%)、 <code>Type.of.Death</code> (42.58%)、 <code>Degree.of.Education</code> (32.86%) |
| 中度 (5%-20%) | 3 | <code>Raca.Color</code> (15.22%)、 <code>Name.Occupation</code> (7.53%) |
| 轻度 ($< 5\%$) | 6 | <code>Nationality</code> (3.03%)、 <code>Age</code> (0.15%) |
| 无缺失 | 15 | <code>Gender</code> 、 <code>year</code> 、 <code>Status.Vital</code> 等 |

9.3 重点字段缺失率

| 字段 | 缺失率 | 临床意义 | 处理建议 |
|---------------------------------|--------|-----------------------|----------------------|
| <code>TNM</code> | 77.15% | 肿瘤分期（T/N/M 组合），核心预后因素 | 缺失太多，直接删除或建"缺失"指示变量 |
| <code>Extension</code> | 60.58% | 肿瘤扩展程度 | 同上 |
| <code>Distant.metastasis</code> | 94.97% | 远处转移 | 几乎全空，建议删除或仅作"有无记录"指示 |
| <code>Age</code> | 0.15% | 年龄 | 缺失极少，可直接删除缺失行或中位数填补 |
| <code>Gender</code> | 0% | 性别 | 完整，无需处理 |

重要发现：`TNM` (77.15%) 和 `Extension` (60.58%) 是肿瘤学中最重要两个分期字段，但缺失率都偏高。这提醒我们：**临床价值高的字段不一定数据质量好**，建模时要在"信息量"和"可用样本量"之间权衡。

十、三种缺失机制：MCAR / MAR / MNAR

缺失值的处理策略高度依赖于缺失机制。统计学家把缺失机制分为三类，理解它们是做好缺失值处理的前提。

10.1 MCAR (Missing Completely At Random, 完全随机缺失)

定义：缺失的发生与**任何变量**（无论已观测还是未观测）都**无关**，完全是随机的。

特点：

- 缺失就像"掷骰子"决定的，没有任何规律。
- 在这种情况下，直接删除缺失样本（listwise deletion）不会引入偏差。
- 现实中**很少见**——真实数据几乎不会是严格的 MCAR。

本数据集示例：`Age` 缺失 0.15%（315 条），可能是极少数录入时手滑漏填，接近 MCAR。

10.2 MAR (Missing At Random, 条件随机缺失)

定义：缺失的发生与**已观测的其他变量**有关，但与**缺失值本身**无关。

注意："At Random" 这个名字很容易误导——MAR **不是**完全随机的！它的意思是"在给定已观测变量的条件下，缺失是随机的"。

特点：

- 缺失有规律，但规律可以被已观测变量"解释"。
- 可以用已观测变量来预测缺失值（如多重插补 Multiple Imputation）。
- 现实中最常见的假设，也是大多数统计方法的默认假设。

本数据集示例： `Degree.of.Education`（教育程度）缺失 32.86%。高龄患者可能更倾向于缺失教育信息（年代久远、记录不全）——如果"年龄"是已观测的，那么教育程度的缺失就属于 MAR。

10.3 MNAR (Missing Not At Random, 非随机缺失)

定义：缺失的发生与**缺失值本身**有关——也就是说，"缺失"这件事取决于"如果填了会是什么值"。

特点：

- 最难处理的一类，因为缺失原因本身不可观测。
- 无法用已观测数据完全"解释"缺失，任何处理都带有假设。
- 处理不当会引入严重偏差。

本数据集示例： `Type.of.Death`（死亡类型）缺失 42.58%。很可能存活患者（VIVO）根本不会有"死亡类型"信息——缺失本身就与生死状态相关。这是典型的 MNAR。

10.4 三种机制对比

| 机制 | 缺失与什么有关 | 能否直接删除 | 处理方法 |
|------|----------|--------|--------------------|
| MCAR | 与任何变量都无关 | 可以，无偏差 | 直接删除 / 均值填补 / 多重插补 |
| MAR | 与已观测变量有关 | 有偏差 | 多重插补 / 基于模型填补 |
| MNAR | 与缺失值本身有关 | 严重偏差 | 需要敏感性分析，无法完全解决 |

重要概念：现实中很难严格区分 MAR 和 MNAR，因为我们看不到缺失的值本身。实践中通常**假设 MAR**（这是最可操作的假设），并在讨论中说明这一假设的局限性。

十一、为什么医学数据经常出现缺失？

结合本数据集，医学数据高缺失的常见原因：

1. **数据来源多样：**不同医院、不同医生记录习惯不同，字段覆盖不一致。
2. **检查项目因病情而异：**并非所有患者都做所有检查。例如早期患者可能不做 `Distant.metastasis`（远处转移）评估。
3. **回顾性数据收集：**历史记录不完整，早期病例的字段比新病例少。
4. **患者隐私考量：**部分敏感字段（如职业、收入）被有意省略。
5. **数据录入错误或遗漏：**人工录入难免出错。

小贴士：在分析缺失时，**不要急于"填补"或"删除"**。先问自己：这个字段为什么缺失？缺失本身是否携带着信息？有时候"是否缺失"本身就是一个有价值的特征（缺失指示变量 missing indicator）。

十二、小贴士与常见问题

小贴士

1. **先统计，再处理**：永远先看缺失比例和模式，再决定处理策略。盲目填补会掩盖数据的问题。
2. **isnull() 和 isna() 等价**：Pandas 中两者完全一样，`isna()` 是后来加的别名，用哪个都行。
3. **抽样时固定种子**：任何涉及随机的过程（抽样、打乱、初始化）都应 `np.random.seed()`，保证可复现。
4. **热力图配色有讲究**：
 - 0/1 离散值 → 传颜色列表 `['#2ecc71', '#e74c3c']`。
 - 相关系数 → `'RdBu_r'` + `center=0`，红正蓝负白为零。
5. **相关性矩阵只画一半**：用 `np.triu(..., k=1)` 生成掩码，避免冗余信息。
6. **缺失指示变量**：对于 MNAR/MAR 的字段，可以新建一列 `XXX_is_missing` (0/1)，让模型自己学习"缺失"的信号。

常见问题

Q1: `df.isnull().sum()` 和 `df.isna().sum()` 有区别吗？

A: 没有区别。两者在 Pandas 中是完全相同的操作，`isna` 是 `isnull` 的别名。习惯上用哪个都可以，社区里两种都很常见。

Q2: 为什么我用 `barh` 画的图，第一个柱子在底部？

A: 因为 Matplotlib 的 y 轴默认从下到上递增。加一行 `ax.invert_yaxis()` 反转 y 轴即可让第一个柱子显示在顶部。

Q3: `sns.heatmap` 的 `cmap` 传列表和传字符串有什么区别？

A: 传字符串（如 `'Blues'`）是传一个 colormap 对象的名字，适合连续值，会根据数值大小在颜色渐变中映射。传列表（如 `['#2ecc71', '#e74c3c']`）适合离散值（如 0/1），列表的第 i 个颜色对应数值 i。本例缺失矩阵只有 0 和 1，所以用列表更直观。

Q4: `np.random.seed(42)` 一定要写吗？不写会怎样？

A: 不写也能运行，但每次运行结果会不同（因为随机数不同），导致图不可复现。在教程、论文、生产代码中，**强烈建议固定种子**，方便他人复现和调试。42 只是约定俗成，用任何整数都可以。

Q5: `missing_matrix.corr()` 计算的是特征值的相关性吗？

A: **不是**。`missing_matrix` 是 0/1 矩阵（1=缺失，0=存在），所以 `corr()` 计算的是**缺失模式**之间的相关性——即"特征 A 缺失"和"特征 B 缺失"是否倾向于同时发生。这与"特征 A 的值"和"特征 B 的值"之间的相关性是两回事。

Q6: 为什么相关性热力图要隐藏上三角？

A: 因为相关性矩阵是对称的（`corr(A,B) == corr(B,A)`），上三角和下三角信息完全重复。隐藏一半可以让图更简洁，避免视觉冗余。主对角线全是 1（自相关），也没有信息量，所以一起隐藏。

Q7: 缺失率超过多少应该直接删除列？

A: 没有绝对标准，但常见经验法则：

- 缺失 **< 5%**：可直接删除缺失行，或简单填补。
- 缺失 **5%-50%**：建议用模型填补（KNN、多重插补），并考虑加缺失指示变量。
- 缺失 **> 50%**：通常建议删除列，或仅保留"是否缺失"作为指示变量（因为值本身信息量太少）。

重要提醒：删除列之前，先想清楚这个字段对业务是否关键。如果某个字段缺失 80% 但对预测至关重要（如 **TNM** 分期），可以考虑：① 保留并加缺失指示变量；② 把"缺失"作为单独的类别；③ 用领域知识辅助填补。

十三、本模块小结

本模块我们完成了以下工作：

1. **统计缺失**：用 `df.isnull().sum()` 计算每列缺失数，转换为百分比，整理成排序表格。
2. **分档归类**：用 `def + if-elif-else` 定义分档函数，用 `.apply()` 应用到 Series，把缺失程度分为"无/轻度/中度/高度/严重"五档。
3. **可视化**：
 - 图 2a：水平条形图展示 Top 30 缺失列，用 `ax.text()` 添加数值标签。
 - 图 2b：缺失模式热力图，抽样 5000 行，用红绿色块直观显示缺失分布。
 - 图 2c：缺失相关性热力图，用上三角掩码去除冗余，用 RdBu_r 配色显示正负相关。
4. **机制讨论**：区分 MCAR/MAR/MNAR 三种缺失机制，结合本数据集给出示例。
5. **关键发现**：24/39 列存在缺失，10 列缺失 $\geq 50\%$ ，`TNM` (77.15%)、`Extension` (60.58%) 等核心临床字段缺失严重。

下一步：在了解了缺失情况后，模块 3 将进入**分布分析**——用直方图、KDE、Q-Q 图研究数值特征的分布形态，为后续的特征工程和建模选择提供依据。

模块 3：分布分析

核心思想：在建模之前，我们必须先"认识"每一个数值特征。它长什么样？是钟形对称的，还是歪向一边？尾巴有多长？有没有离群点？这些信息决定了我们后续是否需要做对数变换、是否需要标准化、是否能用基于正态假设的模型。本模块带你一步步完成分布分析的全流程。

一、学习目标

学完本模块后，你将能够：

1. **理解** `df.select_dtypes()` 按数据类型筛选列的机制，并能用列表推导式排除不需要的列。
2. **计算并解释** 五大描述性统计量：均值、中位数、标准差、偏度、峰度，并写出它们的公式直觉。
3. **判断** 一个特征是近似正态、轻度偏态还是显著偏态，并知道每种情况对建模的影响。
4. **绘制并解读** 三种核心分布图：直方图 + KDE、Q-Q 图、分组箱线图。
5. **理解** 核密度估计（KDE）的原理，以及正态分布概率密度函数（PDF）公式的每个符号含义。
6. **识别** "数值陷阱"——即那些看起来是数值、本质却是类别的变量（如职业代码、形态学代码）。
7. **运用** Matplotlib 的关键参数（`density`、`alpha`、`patch_artist` 等）自定义图表外观。

二、背景回顾

本教程使用的是**巴西癌症登记数据**（Brazilian Cancer Registry）。在模块 1 中，我们已经过滤掉 `Status.Vital`（生存状态）缺失的样本，剩余 **209,758 条**记录。

经过模块 2 的缺失值分析后，数据集中被 pandas 识别为数值型（`np.number`）的特征共有 4 个：

| 特征名 | 含义 | 备注 |
|--------------------|-------------------|---------------|
| Age | 患者诊断时的年龄（岁） | 真正的连续型数值 |
| Code.Profession | 职业分类代码 | 本质是类别变量（数值陷阱） |
| Code.of.Morphology | 肿瘤形态学代码（ICD-O 标准） | 本质是类别变量（数值陷阱） |
| year | 诊断年份（如 2008、2013） | 离散型时间变量 |

重要提醒：被 pandas 识别为"数值" ≠ 真正的连续型数值变量。我们会在本模块末尾详细讨论"数值陷阱"问题。

三、第一步：选取数值型特征

3.1 用 `select_dtypes` 按数据类型筛选列

```
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
```

逐层拆解：

- `df.select_dtypes(include=[np.number])`：这是 pandas DataFrame 的一个方法，作用是**按列的数据类型筛选**，返回一个新的 DataFrame，只包含符合类型的列。
- `include=[np.number]` 表示"只保留数值类型的列"。`np.number` 是 NumPy 中所有数值类型的"总称"，它涵盖了：
 - `int8 / int16 / int32 / int64`（整数）
 - `float16 / float32 / float64`（浮点数）
 - `uint8 / uint16 / uint32 / uint64`（无符号整数）
 - `complex64 / complex128`（复数，极少见）
- 如果你想筛选别的类型，可以这样用：
 - `include=['object']` → 筛选字符串/混合类型列
 - `include=['datetime64']` → 筛选日期时间列
 - `include=['category']` → 筛选类别型列
 - `exclude=[np.number]` → 排除数值列，保留其他所有列
- `.columns`：取出筛选后 DataFrame 的列名（一个 `Index` 对象）。
- `.tolist()`：把 `Index` 对象转换成普通的 Python 列表（`list`），方便后续用列表推导式操作。

小贴士：为什么用 `np.number` 而不是 `'number'`？两者其实等价，pandas 都支持。但用 `np.number` 更明确地表明我们引用的是 NumPy 的类型系统，在团队协作时代码可读性更好。

3.2 用列表推导式排除不需要的列

```
exclude_cols = ['target', 'Patient.Code']
numeric_features = [c for c in numeric_cols if c not in exclude_cols]
```

列表推导式（List Comprehension）语法解析：

```
[<表达式> for <变量> in <可迭代对象> if <条件>]
```

读作："对于 `numeric_cols` 中的每一个列名 `c`，如果 `c` 不在 `exclude_cols` 列表里，就把 `c` 放进新列表。"

为什么要排除这两列？

- `target`：这是我们要预测的目标变量（生存状态）。在分布分析阶段，我们只关心**特征**的分布，不关心目标本身。把目标混进来会污染分析结果。
- `Patient.Code`：这是患者的唯一标识符（ID），本质上是一个“行号”，没有任何统计意义。它的均值、标准差都是无意义的数字。

常见问题：为什么不直接在 `select_dtypes` 里排除？

答：`select_dtypes` 只能按**数据类型**筛选，不能按**列名**筛选。所以需要先使用 `select_dtypes` 拿到所有数值列，再用列表推导式按名字剔除特定列。这是 pandas 工作流中非常常见的“两步走”模式。

运行后，`numeric_features` 的值为：

```
['Age', 'Code.Profession', 'Code.of.Morphology', 'year']
```

四、第二步：计算描述性统计量

接下来，我们对每一个数值特征计算 5 个核心统计量。先看完整代码：

```
dist_results = []
for col in numeric_features:
    data = df[col].dropna()      # 丢弃缺失值
    if len(data) < 50:          # 样本太少则跳过
        continue

    mean_val = data.mean()      # 均值
    median_val = data.median()  # 中位数
    std_val = data.std()        # 标准差
    skewness = data.skew()      # 偏度
    kurtosis_val = data.kurtosis() # 峰度
    ...
```

4.1 `data.dropna()`：丢弃缺失值

在计算统计量之前，必须先丢弃缺失值（NaN）。否则 `mean()`、`std()` 等函数虽然能自动跳过 NaN，但 `len(data)` 会包含缺失值的计数，导致样本量统计错误。`dropna()` 返回一个不含 NaN 的新 Series。

4.2 `if len(data) < 50: continue`：样本量门槛

如果某个特征的有效样本少于 50 条，统计量（尤其是偏度和峰度）会非常不稳定，没有参考价值。所以直接 `continue` 跳过。

4.3 五大统计量详解

(1) `data.mean()` —— 均值 (Mean)

含义：所有数据的算术平均值，反映数据的“中心位置”。

公式直觉：

```
$$
\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i = \frac{x_1 + x_2 + \cdots + x_n}{n}
$$
```

特点：

- 优点：用了所有数据的信息，数学性质好（如最小二乘法的核心）。
- 缺点：**对离群值极其敏感**。一个亿万富翁走进酒吧，所有顾客的"平均资产"瞬间暴涨。

(2) `data.median()` —— 中位数 (Median)

含义：把数据从小到大排序后，位于正中间的那个数。如果数据量是偶数，则取中间两个数的平均值。

公式直觉：

```
$$
\text{Median} = \begin{cases}
x_{\{(n+1)/2\}} & n \text{ 为奇数} \\
\frac{1}{2}(x_{\{n/2\}} + x_{\{n/2+1\}}) & n \text{ 为偶数}
\end{cases}
$$
```

其中 $x_{\{1\}} \leq x_{\{2\}} \leq \dots \leq x_{\{n\}}$ 是排序后的数据。

特点：

- 优点：**对离群值鲁棒 (robust)**。亿万富翁走进酒吧，中位数几乎不变。
- 缺点：只用了中间一两个数据点的信息，忽略了大部分数据。

小贴士：比较均值和中位数，是判断分布偏态的"快速肉眼法"：

- 均值 $\approx$ 中位数 $\rightarrow$ 分布大致对称
- 均值 $>$ 中位数 $\rightarrow$ 右偏（正偏），右侧有长尾
- 均值 $<$ 中位数 $\rightarrow$ 左偏（负偏），左侧有长尾

(3) `data.std()` —— 标准差 (Standard Deviation)

含义：衡量数据围绕均值的"离散程度"。标准差越大，数据越分散；越小，数据越集中。

公式直觉：

```
$$
s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}
$$
```

关键点：

- 分母是 $n-1$ 而不是 n ，这叫**贝塞尔校正 (Bessel's correction)**。原因是：我们用样本均值 $\bar{x}$ 代替了总体均值 μ ，这会低估方差，所以除以 $n-1$ 来修正。pandas 的 `std()` 默认 `ddof=1`，即用 $n-1$ 。
- 标准差的单位与原始数据相同（如 Age 的标准差单位是"岁"），所以比方差（单位是"岁的平方"）更直观。

(4) `data.skew()` —— 偏度 (Skewness)

含义：衡量分布的**不对称程度**。这是本模块最核心的统计量之一。

公式直觉：

```
$$
\text{Skewness} = \frac{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^3}{\left( \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2 \right)^{3/2}}
$$
```

直觉理解：分子是"三阶中心矩"，衡量数据偏离均值后的"立方和"。如果右侧（大于均值）的尾巴长，正的偏离值在立方后会主导，偏度为正；反之偏度为负。

判断标准（本教程采用）：

| 偏度绝对值 | 分布类型 | 说明 |
|---|-----------------------|-------------|
| <code>abs(skewness) < 0.5</code> | 近似正态 (Approx. Normal) | 对称性较好，可视为正态 |
| <code>0.5 ≤ abs(skewness) < 1</code> | 轻度偏态 (Mildly Skewed) | 有点歪，但不太严重 |
| <code>abs(skewness) ≥ 1</code> | 显著偏态 (Highly Skewed) | 严重不对称，需要变换 |

偏度符号的含义：

- 偏度 > 0：**右偏（正偏）**，右侧有长尾，均值 > 中位数。
- 偏度 < 0：**左偏（负偏）**，左侧有长尾，均值 < 中位数。
- 偏度 = 0：完全对称（如正态分布）。

常见误区：很多人以为"右偏"就是"峰在右边"。**恰恰相反！** 右偏分布的峰在**左边**，长尾在**右边**。记忆口诀："偏"指的是**尾巴的方向**，不是**峰的方向**。

对应代码：

```

if abs(skewness) < 0.5:
    dist_type = "近似正态 (Approx. Normal)"
elif abs(skewness) < 1:
    dist_type = "轻度偏态 (Mildly Skewed)"
else:
    dist_type = "显著偏态 (Highly Skewed)"

```

(5) `data.kurtosis()` —— 峰度 (Kurtosis)

含义：衡量分布的"尖锐程度"和"尾巴厚度"。

公式直觉：

$$\text{Kurtosis} = \frac{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^4}{\left(\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2 \right)^2} - 3$$

注意：pandas 的 `kurtosis()` 默认返回的是**超额峰度 (excess kurtosis)**，即"原始峰度减 3"。这样做的目的是让正态分布的峰度等于 0，方便比较。

判断标准：

| 超额峰度 | 分布形态 | 说明 |
|------|---------------------------|--------------------|
| > 0 | 尖峰厚尾 (Leptokurtic) | 比正态分布更尖、尾巴更厚，极端值更多 |
| = 0 | 正态 (Mesokurtic) | 与正态分布相同 |
| < 0 | 平峰薄尾 (Platykurtic) | 比正态分布更平、尾巴更薄，极端值更少 |

小贴士：峰度大意味着"厚尾"，这在金融风控中极其重要——厚尾分布意味着"黑天鹅"事件（极端损失）比正态分布预测的更频繁。2008 年金融危机的部分原因，就是风险模型错误假设了正态分布（薄尾）。

4.4 把结果存入 DataFrame

```
dist_results.append({
    'Feature': col,
    'Count': len(data),
    'Mean': mean_val,
    'Median': median_val,
    'Std': std_val,
    'Skewness': skewness,
    'Kurtosis': kurtosis_val,
    'Distribution': dist_type
})

dist_df = pd.DataFrame(dist_results)
```

这里我们用一个列表 `dist_results` 收集每个特征的统计字典，最后一次性转成 DataFrame。这比"逐行 append 到 DataFrame"高效得多，是 pandas 的最佳实践。

五、第三步：图 3a —— 直方图 + KDE + 正态曲线

5.1 代码全貌

```
plot_features = ['Age'] if 'Age' in numeric_features else numeric_features[:2]

fig, axes = plt.subplots(1, len(plot_features), figsize=(6 * len(plot_features), 5))
if len(plot_features) == 1:
    axes = [axes]

for ax, col in zip(axes, plot_features):
    data = df[col].dropna()
    if len(data) > 50000:
        data_sample = data.sample(50000, random_state=42)
    else:
        data_sample = data

    ax.hist(data_sample, bins=80, density=True, alpha=0.6,
            color='#3498db', edgecolor='white', label='Histogram')
    data_sample.plot.kde(ax=ax, color='#e74c3c', linewidth=2.5, label='KDE')
    x_range = np.linspace(data_sample.min(), data_sample.max(), 1000)
    normal_pdf = (1 / (data_sample.std() * np.sqrt(2 * np.pi))) * \
        np.exp(-0.5 * ((x_range - data_sample.mean()) / data_sample.std()) ** 2)
    ax.plot(x_range, normal_pdf, '--', color='#2c3e50', linewidth=1.5,
            label=f'N(μ={data_sample.mean():.1f}, σ={data_sample.std():.1f})')
```

5.2 三元表达式选取绘图特征

```
plot_features = ['Age'] if 'Age' in numeric_features else numeric_features[:2]
```

这是一个**三元条件表达式**：如果 `Age` 在数值特征里，就只画 `Age`；否则取前两个数值特征。优先选 `Age` 是因为它是本数据集中**唯一真正的连续型数值变量**，最适合展示分布分析。

5.3 处理"单子图"的陷阱

```
fig, axes = plt.subplots(1, len(plot_features), figsize=(6 * len(plot_features), 5))
if len(plot_features) == 1:
    axes = [axes]
```

为什么要 `axes = [axes]` ?

当 `plt.subplots(1, 1)` 创建 1×1 子图时, 返回的 `axes` 是单个 **Axes 对象**, 不是列表。但 `for ax, col in zip(axes, plot_features)` 需要 `axes` 是可迭代的。所以用 `axes = [axes]` 把它包成列表, 保证循环能正常工作。这是 Matplotlib 中极其常见的"坑"。

5.4 大数据集抽样

```
if len(data) > 50000:
    data_sample = data.sample(50000, random_state=42)
else:
    data_sample = data
```

为什么要抽样?

- **Age** 有 209,443 条数据, 全部画直方图会非常慢, KDE 计算更是 $O(n^2)$ 复杂度。
- 50,000 条已经足够稳定地估计分布形状。
- `random_state=42` 保证每次抽样结果一致, 便于复现。42 是程序员的"梗" (来自《银河系漫游指南》中"生命、宇宙和一切的答案")。

5.5 `ax.hist()` 参数详解

```
ax.hist(data_sample, bins=80, density=True, alpha=0.6,
        color='#3498db', edgecolor='white', label='Histogram')
```

| 参数 | 值 | 作用 |
|--------------------------|-------------|---|
| <code>data_sample</code> | 输入数据 | 要绘制直方图的数据 |
| <code>bins</code> | 80 | 把数据范围等分成 80 个区间 ("桶")。 <code>bins</code> 越多, 直方图越细, 但太大会被噪声主导; 太少则掩盖结构。经验法则: $\text{bins} \approx \sqrt{n}$, 这里 $\sqrt{50000} \approx 224$, 但 80 已经足够清晰 |
| <code>density</code> | True | 关键参数! 把纵轴从"频数 (count)"归一化为"概率密度 (density)", 使直方图总面积 = 1。这样才能和 KDE 曲线、正态 PDF 曲线画在同一坐标系下比较 |
| <code>alpha</code> | 0.6 | 透明度, 0=全透明, 1=不透明。设为 0.6 是为了能透过直方图看到下方的曲线 |
| <code>color</code> | '#3498db' | 直方图的填充色 (蓝色, 十六进制 RGB) |
| <code>edgecolor</code> | 'white' | 每个柱子的边框颜色。白色边框让相邻柱子视觉上更易区分 |
| <code>label</code> | 'Histogram' | 图例标签 |

常见问题: 为什么用 `density=True` 而不是 `frequency` (频数) ?

因为我们要把直方图、KDE 曲线、正态 PDF 曲线画在同一张图上比较。后两者都是"概率密度", 单位是"1/单位"。如果直方图用频数 (如 500、1000), 纵轴量纲不一致, 曲线会被压扁到几乎看不见。`density=True` 把直方图归一化为"概率密度", 三者量纲统一, 可以直接叠加。

5.6 KDE (核密度估计) 原理详解

```
data_sample.plot.kde(ax=ax, color='#e74c3c', linewidth=2.5, label='KDE')
```

什么是 KDE (Kernel Density Estimation, 核密度估计) ?

KDE 是一种**非参数**方法, 用于从一组样本数据中估计其背后的概率密度函数 (PDF)。直方图是"分桶计数", KDE 则是"每个数据点贡献一个小山丘, 叠加起来形成平滑曲线"。

数学公式:

$$\hat{f}(x) = \frac{1}{n h} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right)$$

逐项解释:

- $\hat{f}(x)$: 在点 x 处估计的概率密度。
- n : 样本量。
- h : **带宽 (bandwidth)**, 控制"小山丘"的宽度。 h 越大, 曲线越平滑但可能掩盖细节; h 越小, 曲线越尖锐但可能过拟合噪声。
- $K(\cdot)$: **核函数 (kernel function)**, 通常用高斯核 (正态分布的形状)。它决定了每个数据点贡献的"小山丘"长什么样。
- x_i : 第 i 个数据点。

直觉理解:

想象每个数据点都"点亮"一盏小灯, 灯光呈高斯分布向四周衰减。所有灯的光叠加起来, 就形成了一条平滑的密度曲线。带宽 h 就是"灯的亮度范围"。

KDE vs 直方图:

| | | |
|------|------------------|-----------------------|
| 特性 | 直方图 | KDE |
| 形态 | 阶梯状 (分桶) | 平滑曲线 |
| 依赖参数 | bins (桶数) | bandwidth (带宽) |
| 边界效应 | 有明显的桶边界 | 无桶边界, 但边界处可能泄漏 |
| 计算成本 | 低 | 较高 ($O(n^2)$) |

5.7 正态分布 PDF 公式详解

```
x_range = np.linspace(data_sample.min(), data_sample.max(), 1000)
normal_pdf = (1 / (data_sample.std() * np.sqrt(2 * np.pi))) * \
    np.exp(-0.5 * ((x_range - data_sample.mean()) / data_sample.std()) ** 2)
ax.plot(x_range, normal_pdf, '--', color='#2c3e50', linewidth=1.5,
    label=f'N(μ={data_sample.mean():.1f}, σ={data_sample.std():.1f})')
```

正态分布概率密度函数 (PDF) 公式:

$$f(x) = \frac{1}{\sigma \sqrt{2\pi}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

每个符号的含义:

| 符号 | 含义 | 代码对应 |
|--------------------------------|-----------------------|---|
| x | 自变量（横轴的值） | <code>x_range</code> |
| μ | 均值（决定钟形曲线的中心位置） | <code>data_sample.mean()</code> |
| σ | 标准差（决定钟形曲线的宽窄） | <code>data_sample.std()</code> |
| π | 圆周率 ≈ 3.14159 | <code>np.pi</code> |
| $\exp(\cdot)$ | 自然指数函数 $e^{(\cdot)}$ | <code>np.exp(...)</code> |
| $\frac{1}{\sigma\sqrt{2\pi}}$ | 归一化常数，保证曲线下面积 = 1 | <code>1 / (data_sample.std() * np.sqrt(2 * np.pi))</code> |
| $-\frac{(x-\mu)^2}{2\sigma^2}$ | 指数部分，决定钟形 | <code>-0.5 * ((x_range - mean) / std) ** 2</code> |

直觉理解：

- μ 决定"钟"的中心在哪里。
- σ 决定"钟"有多宽。 σ 越大，钟越矮胖； σ 越小，钟越瘦高。
- 指数部分 $-\frac{(x-\mu)^2}{2\sigma^2}$ 始终 ≤ 0 ，所以 $\exp$ 后 ≤ 1 。当 $x = \mu$ 时取最大值 1，对应峰值 $\frac{1}{\sigma\sqrt{2\pi}}$ 。
- 我们画这条曲线，是为了**直观对比**：如果数据真的服从正态分布，KDE 曲线应该和这条虚线重合。偏离越多，说明越不正态。

5.8 `np.linspace` 生成等差数列

```
x_range = np.linspace(data_sample.min(), data_sample.max(), 1000)
```

`np.linspace(start, stop, num)` 生成从 `start` 到 `stop` 的 `num` 个等间距点（包含两端）。

示例：

```
np.linspace(0, 10, 5) # → array([ 0. ,  2.5,  5. ,  7.5, 10. ])
```

为什么用 1000 个点？

画曲线需要足够多的点才能显得平滑。1000 个点足以让正态 PDF 曲线看起来非常顺滑，而计算成本可以忽略。

小贴士： `np.linspace` 和 `np.arange` 的区别：

- `linspace(0, 10, 5)` → 指定**点数**，自动算间距，**包含终点**。
- `arange(0, 10, 2.5)` → 指定**步长**，自动算点数，**不包含终点**。

画曲线时优先用 `linspace`，因为你能精确控制采样密度。

5.9 结果图

分布直方图+KDE

如何解读这张图？

- **蓝色直方图：** Age 的实际分布。可以看到峰值在 65 岁左右，左侧（低龄）上升较缓，右侧（高龄）下降较快，整体略微左偏。
- **红色 KDE 曲线：** 平滑后的密度估计，与直方图形状吻合。
- **黑色虚线：** 以样本均值和标准差为参数的正态分布曲线。对比可见，Age 分布比正态分布略尖，左尾稍长，印证了偏度 $= -0.683$ （轻度左偏）的结论。

六、第四步：图 3b —— Q-Q 图（分位数图）

6.1 代码全貌

```
from scipy import stats
fig, axes = plt.subplots(1, len(plot_features), figsize=(6 * len(plot_features), 5))
if len(plot_features) == 1:
    axes = [axes]

for ax, col in zip(axes, plot_features):
    data = df[col].dropna()
    if len(data) > 5000:
        data_qq = data.sample(5000, random_state=42)
    else:
        data_qq = data

    stats.probplot(data_qq, dist="norm", plot=ax)
    ax.get_lines()[0].set_markerfacecolor('#3498db')
    ax.get_lines()[1].set_color('#e74c3c')
```

6.2 Q-Q 图原理详解

Q-Q 图（Quantile-Quantile Plot，分位数-分位数图） 是判断数据是否服从某分布（通常是正态分布）的最直观工具。

核心思想：分位数对分位数。

具体步骤：

1. 把样本数据从小到大排序： $x_{(1)} \leq x_{(2)} \leq \dots \leq x_{(n)}$ 。
2. 对每个 $x_{(i)}$ ，计算它的“经验分位数”位置 $p_i = \frac{i - 0.5}{n}$ （用 0.5 是避免 $p=0$ 或 $p=1$ 时理论分位数为无穷大）。
3. 在标准正态分布 $N(0,1)$ 中，找到分位数 $z_i = \Phi^{-1}(p_i)$ （即标准正态分布的 p_i 分位数）。
4. 在坐标系中画散点：横轴是 z_i （理论分位数），纵轴是 $x_{(i)}$ （样本分位数）。

如何判断正态性？

- **如果数据服从正态分布**：所有点会落在一条直线上（这条直线就是“参考线”）。
- **如果数据有重尾（厚尾）**：两端的点会偏离参考线，呈“S 形”或“反 S 形”。
- **如果数据右偏**：右上端的点会向上翘起（高于参考线）。
- **如果数据左偏**：左下端的点会向下垂落（低于参考线）。

小贴士： Q-Q 图比偏度数值更直观。偏度只给一个数字，Q-Q 图能让你看到“哪里偏离了正态”——是中间、是两端、还是某一侧。

6.3 stats.probplot 参数详解

```
stats.probplot(data_qq, dist="norm", plot=ax)
```

| 参数 | 值 | 作用 |
|----------------------|---------------------|--|
| <code>data_qq</code> | 输入数据 | 要检验的样本数据（一维数组） |
| <code>dist</code> | <code>"norm"</code> | 假设的理论分布。 <code>"norm"</code> 是正态分布；也可以是 <code>"expon"</code> （指数）、 <code>"lognorm"</code> （对数正态）、 <code>"uniform"</code> （均匀）等 |
| <code>plot</code> | <code>ax</code> | 把图绘制到指定的 Matplotlib Axes 对象上。如果不传，则不画图，只返回计算结果 |

返回值： `probplot` 会返回一个元组 `(osm, osr)`，其中 `osm` 是理论分位数，`osr` 是样本分位数。但因为我们传了 `plot=ax`，它会自动画图。

6.4 `ax.get_lines()[0]` 和 `[1]` 分别是什么？

```
ax.get_lines()[0].set_markerfacecolor('#3498db') # 数据点
ax.get_lines()[1].set_color('#e74c3c')          # 参考线
```

`stats.probplot` 在 Axes 上画了两条 Line2D 对象：

| 索引 | 对应内容 | 默认外观 |
|--------------------------------|-----------|------|
| <code>ax.get_lines()[0]</code> | 样本数据点（散点） | 蓝色圆点 |
| <code>ax.get_lines()[1]</code> | 参考线（理论直线） | 红色实线 |

为什么是这两条？

- 数据点：每个点代表“样本分位数 vs 理论分位数”。
- 参考线：如果数据完全服从正态分布，所有数据点都会落在这条线上。这条线通常用数据的均值和标准差（或最小二乘拟合）确定。

我们通过 `set_markerfacecolor` 把数据点改成蓝色，通过 `set_color` 把参考线改成红色，让配色和图 3a 保持一致。

常见问题：为什么 Q-Q 图抽样只用 5000 个点，而直方图用 50000 个？

答：Q-Q 图是散点图，点太多会糊成一团，反而看不清偏离模式。5000 个点足够展示分布形状，又不会过于密集。直方图是分桶统计，需要更多数据才能让每个桶内的计数稳定。

6.5 结果图

Q-Q图

如何解读这张 Q-Q 图？

- 如果 Age 完全服从正态分布，所有蓝色点都会落在红色参考线上。
- 实际观察：中间部分大致贴合参考线，但**左下端**（低龄端）的点偏离参考线向下，**右上端**（高龄端）的点偏离参考线向下。这种“两端都向下弯”的形态，说明 Age 分布比正态分布**尾部更轻**（薄尾），同时整体略左偏，与偏度 = -0.683 的结论一致。

七、第五步：图 3c —— 按目标分组的箱线图

7.1 代码全貌

```
if 'Age' in df.columns:
    fig, ax = plt.subplots(figsize=(8, 6))
    age_data = df[['Age', 'target']].dropna()
    if len(age_data) > 50000:
        age_data = age_data.sample(50000, random_state=42)

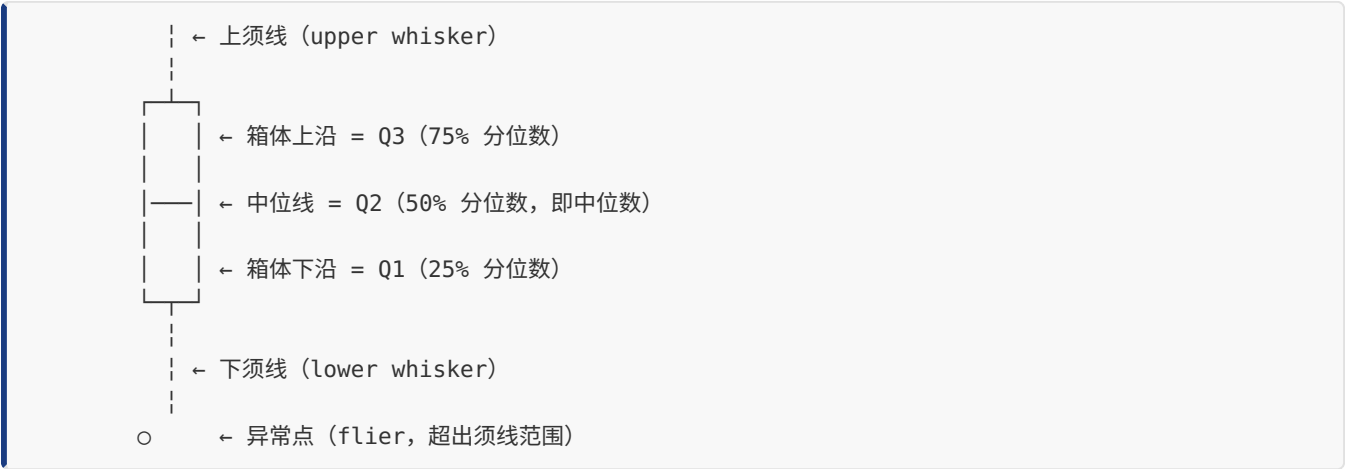
    bp = ax.boxplot([age_data.loc[age_data['target'] == 1, 'Age'].values,
                     age_data.loc[age_data['target'] == 0, 'Age'].values],
                    tick_labels=['VIVO (Alive)', 'MORTO (Dead)'],
                    patch_artist=True,
                    widths=0.4)

    bp['boxes'][0].set_facecolor('#2ecc71')
    bp['boxes'][1].set_facecolor('#e74c3c')
```

7.2 箱线图 (Boxplot) 构成详解

箱线图由五数概括 (Five-number summary) 构成，能一图展示分布的中心、离散、偏态和离群值。

箱线图的五大组成部分：



各部分含义：

| 部分 | 含义 | 计算方式 |
|---------------------|--------------|---|
| 箱体 (box) | 中间 50% 数据的范围 | 从 Q1 到 Q3 |
| 箱体高度 | 四分位距 IQR | $IQR = Q3 - Q1$ |
| 中位线 (median) | 数据的中位数 | Q2 |
| 上须线 (upper whisker) | "正常"数据的上界 | $\min(Q3 + 1.5 \times IQR, \text{数据最大值})$ |
| 下须线 (lower whisker) | "正常"数据的下界 | $\max(Q1 - 1.5 \times IQR, \text{数据最小值})$ |
| 异常点 (fliers) | 超出须线范围的点 | $< Q1 - 1.5 \times IQR$ 或 $> Q3 + 1.5 \times IQR$ |

小贴士： $1.5 \times IQR$ 是 Matplotlib 默认的异常点判定阈值。这个阈值来自 John Tukey (箱线图发明者) 的经验：对于正态分布， $1.5 \times IQR$ 大约对应 $\pm 2.7\sigma$ ，覆盖了 99.3% 的数据，剩下的 0.7% 被标记为异常点。

7.3 patch_artist=True 参数

```
bp = ax.boxplot(..., patch_artist=True, widths=0.4)
```

| 参数 | 作用 |
|--------------|--|
| patch_artist | 默认 <code>False</code> ，箱体只有边框，没有填充色。设为 <code>True</code> 后，箱体变成 <code>Patch</code> 对象，可以填充颜色。这是自定义箱线图外观的关键开关 |
| widths | 箱体的宽度 (0~1 之间)。0.4 是一个适中值，既不会太胖显得笨重，也不会太瘦难以看清 |
| tick_labels | 每个箱子的标签 (之前版本叫 <code>labels</code> ，新版已更名) |

7.4 bp 字典的五个键

`ax.boxplot()` 返回一个字典 `bp`，包含箱线图的所有图形组件：

| 键 | 对应部分 | 类型 | 数量 |
|-----------------------------|-----------------|-----------------|-----------------|
| <code>bp['boxes']</code> | 箱体（Q1 到 Q3 的矩形） | Line2D/Patch 列表 | 每组 1 个 |
| <code>bp['whiskers']</code> | 须线（上下两条） | Line2D 列表 | 每组 2 个（上须、下须） |
| <code>bp['caps']</code> | 须线端帽（须线末端的横线） | Line2D 列表 | 每组 2 个（上帽、下帽） |
| <code>bp['medians']</code> | 中位线（箱体内的横线） | Line2D 列表 | 每组 1 个 |
| <code>bp['fliers']</code> | 异常点（超出须线的点） | Line2D 列表 | 每组 1 个（包含所有异常点） |

对应代码：

```
bp['boxes'][0].set_facecolor('#2ecc71') # 第一个箱体（VIVO）填充绿色
bp['boxes'][1].set_facecolor('#e74c3c') # 第二个箱体（MORTO）填充红色
for whisker in bp['whiskers']:
    whisker.set_color('gray') # 所有须线设为灰色
for cap in bp['caps']:
    cap.set_color('gray') # 所有端帽设为灰色
for median in bp['medians']:
    median.set_color('black') # 所有中位线设为黑色
    median.set_linewidth(2) # 中位线加粗
```

配色逻辑：

- 绿色（`#2ecc71`）→ VIVO（存活），象征“生”。
- 红色（`#e74c3c`）→ MORTO（死亡），象征“亡”。
- 这种“语义化配色”让读者一眼就能理解颜色含义，比随机配色专业得多。

7.5 布尔索引 + `.values`

```
age_data.loc[age_data['target'] == 1, 'Age'].values
```

逐层拆解：

1. `age_data['target'] == 1`：这是一个布尔 Series，对每一行判断 `target` 是否等于 1。结果形如 `[True, False, True, ...]`。
2. `age_data.loc[布尔Series, 'Age']`：`.loc[行选择, 列选择]` 是 pandas 的标签索引。这里“行选择”是布尔 Series，表示“只保留 True 对应的行”；“列选择”是 `'Age'`，表示“只取 Age 列”。结果是一个 Series，包含所有 `target==1` 患者的年龄。
3. `.values`：把 Series 转换成 NumPy 数组。`ax.boxplot()` 需要传入数组或列表的列表，所以用 `.values` 取出底层数据。

常见问题： `.loc` 和 `.iloc` 的区别？

- `.loc`：按**标签**索引，如 `df.loc[行标签, 列标签]`。
 - `.iloc`：按**位置**索引，如 `df.iloc[0:5, 2]`（取前 5 行第 3 列）。
- 布尔索引通常用 `.loc`，因为它能精确匹配标签语义。

7.6 结果图

年龄分组箱线图

如何解读这张箱线图？

- **VIVO（存活）组**：中位数偏低，箱体偏下，说明存活患者整体更年轻。
- **MORTO（死亡）组**：中位数偏高，箱体偏上，说明死亡患者整体更年长。

- 这与医学常识一致：**年龄是癌症生存的重要预后因素**，年龄越大，死亡风险越高。
- 两组箱体有重叠，说明 Age 单独不足以完全区分生死，但确实有显著趋势。后续模块的统计检验会量化这种差异。

八、实际运行结果

下表是 4 个数值特征的实际统计结果（数据集已过滤掉 `Status.Vital` 缺失样本，剩余 209,758 条记录）：

| 特征 | 样本量 | 均值 | 中位数 | 标准差 | 偏度 | 峰度 | 分布类型 |
|--------------------|---------|-----------|-----------|----------|--------|--------|------|
| Age | 209,443 | 63.79 | 65.00 | 16.55 | -0.683 | 0.639 | 轻度偏态 |
| Code.Profession | 193,966 | 218.34 | 0.00 | 289.05 | 0.940 | -0.581 | 轻度偏态 |
| Code.of.Morphology | 209,758 | 82,428.77 | 80,903.00 | 4,468.45 | 2.634 | 6.027 | 显著偏态 |
| year | 209,758 | 2,012.24 | 2,013.00 | 3.92 | -1.004 | 0.786 | 显著偏态 |

8.1 逐特征解读

Age（年龄）

- **样本量** 209,443（比总数少 315，说明有少量缺失）。
- **均值 63.79 < 中位数 65.00** → 左偏（负偏），与偏度 -0.683 一致。
- **偏度绝对值 0.683** 落在 [0.5, 1) 区间 → 轻度偏态。
- **峰度 0.639 > 0** → 比正态分布略尖，有轻微厚尾。
- **结论**：Age 接近正态但略左偏，多数模型可以直接使用，无需变换。

Code.Profession（职业代码）

- **样本量** 193,966（缺失约 15,792 条，缺失率 7.5%）。
- **中位数 = 0**，说明超过一半患者的职业代码是 0（很可能是“未填写”或“无业”的占位符）。
- **均值 218.34 >> 中位数 0** → 强烈右偏。
- **偏度 0.940** 接近 1 → 轻度偏态（临界显著偏态）。
- **数值陷阱！** 这是职业分类代码，不是连续数值。它的“均值 218.34”毫无意义——你不能说“平均职业是 218 号”。详见第八节。

Code.of.Morphology（形态学代码）

- **样本量** 209,758（无缺失）。
- **均值 82,428.77 > 中位数 80,903** → 右偏。
- **偏度 2.634** 远超 1 → **显著偏态**。
- **峰度 6.027** 远大于 0 → **尖峰厚尾**，存在极端值。
- **数值陷阱！** 这是 ICD-O（国际疾病分类-肿瘤学）的形态学代码，如 8090/3 表示“基底细胞癌，恶性”。代码本身是数字，但数字大小没有意义，8090 并不“大于”8080。

year（诊断年份）

- **样本量** 209,758（无缺失）。
- **均值 2012.24 < 中位数 2013** → 左偏。
- **偏度 -1.004** 刚好超过 1 → 显著偏态。
- **峰度 0.786** → 略尖。
- **解读**：数据集中在 2008–2017 年，但 2013 年后样本更多，所以左偏（左侧 2008 年附近的尾巴更长）。year 是离散时间变量，分布偏态本身不影响建模，但要注意**时间漂移**问题（未来年份的分布可能变化）。

九、关键概念：数值陷阱（Numeric Trap）

数值陷阱：某些变量在数据表中被存储为数字（int 或 float），pandas 会自动把它们识别为数值型，但它们**本质上是类别变量**。对它们计算均值、标准差、偏度是**毫无意义**的，甚至会产生误导。

9.1 本数据集中的两个"陷阱"

(1) `Code.Profession`（职业代码）

- 这是一个**分类编码**，每个数字代表一种职业类别（如 0=未填写，100=医生，200=工程师……）。
- 数字大小没有顺序意义——"200 号职业"并不比"100 号职业"多两倍什么。
- **正确处理：**当作类别变量，用 `astype('category')` 或 One-Hot 编码。
- **错误处理：**当作连续数值喂给线性模型，模型会强行拟合一个"职业代码每增加 1，风险变化 X"的关系，这完全是胡说八道。

(2) `Code.of.Morphology`（形态学代码）

- 这是 **ICD-O-3**（International Classification of Diseases for Oncology, 3rd Edition）的形态学编码。
- 编码格式通常是 `M-XXXX/Y`，如 `M-8090/3`（基底细胞癌，恶性）。在数据表中常被存为数字 80903。
- 数字大小没有意义——8090 并不"大于"8080，它们只是不同的肿瘤类型标签。
- **正确处理：**当作类别变量。由于类别数很多（数百个），可以考虑：
 - 只保留高频类别，低频合并为"其他"。
 - 用目标编码（Target Encoding）。
 - 用嵌入层（Embedding，适用于深度学习）。

9.2 如何识别数值陷阱？

| 检查项 | 真正的数值变量 | 伪装的类别变量 |
|------|---------------------|-----------------|
| 单位 | 有明确单位（岁、kg、元） | 无单位，只是代码 |
| 大小关系 | $5 < 10 < 20$ 有物理意义 | 5 和 10 只是不同标签 |
| 小数 | 可以有小数（如 63.5 岁） | 通常是整数 |
| 均值意义 | "平均年龄 63.79 岁"有意义 | "平均职业 218 号"无意义 |
| 取值数量 | 通常很多（连续） | 可能有限（离散类别） |
| 领域知识 | 物理量、统计量 | 编码、ID、等级 |

小贴士：在 EDA 阶段，**永远先看数据字典（data dictionary）和领域知识**，不要盲目相信 pandas 的 `select_dtypes`。一个负责任的数据科学家会把 `Code.Profession` 和 `Code.of.Morphology` 从数值特征列表中剔除，移到类别特征列表中。

9.3 修正后的特征分类

| 特征 | pandas 类型 | 真实类型 | 正确处理方式 |
|---------------------------------|-----------|-----------|----------------|
| <code>Age</code> | float | 连续数值 | 标准化、可直接用于线性模型 |
| <code>year</code> | int | 离散数值/时间 | 当作序数或时间特征 |
| <code>Code.Profession</code> | float | 类别 | One-Hot / 目标编码 |
| <code>Code.of.Morphology</code> | float | 类别 | 高基数类别编码 |

十、常见问题 (FAQ)

Q1: 偏度和峰度的阈值是怎么定的? 为什么是 0.5 和 1?

A: 这些是经验阈值, 不是严格的统计标准。不同教材和领域略有差异:

- 本教程采用: $|\text{skew}| < 0.5$ 近似正态, < 1 轻度偏态, ≥ 1 显著偏态。
- 有的文献用更严格的标准: $|\text{skew}| < 0.5$ 才算对称。
- 有的用更宽松: $|\text{skew}| < 2$ 都算"可接受"。

关键不是阈值本身, 而是**根据偏度决定后续操作**: 显著偏态的特征可能需要对数变换或 Box-Cox 变换。

Q2: 为什么 `density=True` 后直方图纵轴超过 1? 概率密度不是应该 ≤ 1 吗?

A: **概率密度 (density) 和概率 (probability) 是两回事!**

- 概率密度 $f(x)$ 可以大于 1, 只要 $\int f(x)dx = 1$ (总面积为 1)。
- 例如, 均匀分布 $U(0, 0.1)$ 的密度是 $f(x) = 10$ (在 $[0, 0.1]$ 区间内), 但概率不会超过 1。
- 直方图用 `density=True` 后, 纵轴是"每单位的概率", 柱子的高度 $\times$ 柱子的宽度 = 该区间的概率。

Q3: Q-Q 图上的点不呈直线, 但偏度接近 0, 这是为什么?

A: 偏度只衡量**三阶矩** (不对称性), Q-Q 图展示的是**整体分布形状**。可能的情况:

- 分布对称但**厚尾** (如 t 分布): 偏度 ≈ 0 , 但 Q-Q 图两端会偏离参考线。
- 分布对称但**双峰**: 偏度 ≈ 0 , 但 Q-Q 图会呈"S 形"。
- 所以 Q-Q 图比单一偏度数值提供更多信息, 两者应结合使用。

Q4: `data.skew()` 和 `data.kurtosis()` 默认会自动跳过 NaN 吗?

A: **会的**。pandas 的大多数统计方法 (`mean`、`std`、`skew`、`kurtosis`) 默认 `skipna=True`, 会自动忽略 NaN。但为了**精确控制样本量** (如 `len(data)`), 本教程先 `dropna()` 再计算, 这样 `len(data)` 反映的是有效样本数。

Q5: 箱线图的异常点和模块 4 的 IQR 离群值是一回事吗?

A: **基本是一回事**。箱线图默认用 $1.5 \times \text{IQR}$ 规则标记异常点, 这与模块 4 的 IQR 方法一致。但箱线图只是"标记"异常点, 不做删除; 模块 4 会进一步讨论如何处理 (删除、盖帽、变换等)。

Q6: 为什么 `Code.Profession` 的中位数是 0? 0 是有效值还是缺失?

A: 在本数据集中, 0 很可能是**"未填写"或"无业"的占位符**。这体现了真实数据的复杂性: 缺失值不一定表现为 `NaN`, 也可能用 0、-1、999 等特殊值表示。在 EDA 阶段要结合领域知识和数据字典判断。

十一、小贴士汇总

- **小贴士 1:** 用 `df.select_dtypes(include=[np.number])` 筛选数值列后, **一定要人工复核**, 剔除伪装成数值的类别变量 (如 ID、编码)。
- **小贴士 2:** 比较均值和中位数是判断偏态的"快速肉眼法": 均值 $>$ 中位数 $\rightarrow$ 右偏; 均值 $<$ 中位数 $\rightarrow$ 左偏。
- **小贴士 3:** 画直方图时, `bins` 不是越多越好。经验法则 $\text{bins} \approx \sqrt{n}$, 或用 Freedman-Diaconis 规则。pandas 的 `hist()` 默认用 30, 可手动调整。
- **小贴士 4:** KDE 的带宽 `h` 对结果影响巨大。pandas 默认用 Scott 规则。如果曲线太平滑或太抖, 可以手动指定 `bw_method`。
- **小贴士 5:** Q-Q 图抽样时, 5000 个点足够。点太多会糊成一团, 反而看不清偏离模式。
- **小贴士 6:** 箱线图的 `patch_artist=True` 是自定义配色的关键开关, 默认 `False` 时箱体无法填充颜色。
- **小贴士 7:** **永远不要**对类别编码 (如职业代码、邮编) 计算均值和标准差。在 EDA 报告中, 这类变量应移到"类别特征"章节分析。

小贴士 8: `random_state=42` 不是必须的，但保证结果可复现。在教程和论文中，固定随机种子是基本规范。

十二、本模块小结

本模块我们完成了：

1. **选取数值特征**：用 `select_dtypes` 按类型筛选，用列表推导式排除 `target` 和 `Patient.Code`。
2. **计算五大统计量**：均值、中位数、标准差、偏度、峰度，并理解了每个统计量的公式直觉和适用场景。
3. **判断分布类型**：基于偏度绝对值，分为近似正态、轻度偏态、显著偏态三类。
4. **绘制三种核心分布图**：
 - **直方图 + KDE + 正态曲线**：直观对比实际分布与正态分布。
 - **Q-Q 图**：通过分位数对分位数，精确判断正态性。
 - **分组箱线图**：按目标变量分组，观察特征在不同类别下的分布差异。
5. **识别数值陷阱**：`Code.Profession` 和 `Code.of.Morphology` 虽然是数值类型，但本质是类别变量，不能当作连续数值处理。

下一步：在模块 4 中，我们将基于本模块的分布分析，进一步用 IQR 和 Z-score 方法识别和处理离群值。

延伸阅读：

- pandas 官方文档：[select_dtypes](#)
- scipy 官方文档：[stats.probplot](#)
- Matplotlib 官方文档：[boxplot](#)
- 维基百科：[核密度估计](#)、[Q-Q 图](#)

模块 4：离群值分析

核心思想：离群值（Outlier）是数据集中明显偏离其他观测值的数据点。它们可能是真实的极端现象，也可能是测量或录入错误。识别离群值是 EDA 中至关重要的一步——既不能盲目删除（可能丢失重要信息），也不能视而不见（可能污染模型）。

本模块将以巴西癌症登记数据为例，介绍两种最常用的离群值检测方法：**IQR（四分位距）方法** 和 **Z-score（标准分）方法**，并通过可视化对比它们的差异。

学习目标

完成本模块后，你将能够：

1. **理解 IQR 方法的原理**：掌握 Q1、Q3、IQR 的含义，以及 $1.5 \times \text{IQR}$ 界限的由来。
2. **理解 Z-score 方法的原理**：掌握标准化变换 $Z = (x - \mu) / \sigma$ 的含义，以及 3σ 原则。
3. **对比两种方法的优缺点**：知道在什么场景下选择哪种方法。
4. **使用 matplotlib 绘制箱线图和散点图**：理解 `boxplot`、`scatter`、`axhline`、`colorbar` 等关键参数。
5. **使用 pandas 的 melt 进行宽表转长表**：理解为什么 seaborn 通常需要“长格式”数据。
6. **使用 seaborn 绘制多变量箱线图**：理解 `hue` 和 `legend` 参数在 seaborn 0.13+ 中的新用法。
7. **批判性思考医学异常值**：理解“离群值 $\neq$ 错误值”，学会结合领域知识判断。

1. 背景与数据准备

1.1 数据集回顾

我们使用的是巴西癌症登记数据，已经过滤掉 `Status.Vital` 缺失的样本，剩余 **209,758 条记录**。

数值特征共有 4 个：

| 特征名 | 含义 | 备注 |
|--------------------|----------------|---------------------|
| Age | 患者确诊年龄 | 部分样本缺失（209,443 条有效） |
| Code.Profession | 职业编码 | 缺失较多（193,966 条有效） |
| Code.of.Morphology | 形态学编码（ICD-O-3） | 完整 |
| year | 确诊年份 | 完整 |

1.2 代码框架

离群值分析的整体逻辑是：遍历每个数值特征 → 计算两种方法的离群值 → 汇总结果 → 可视化对比。

```
outlier_features = [f for f in numeric_features if f in df.columns]
outlier_results = []

for col in outlier_features:
    data = df[col].dropna()
    if len(data) < 50:
        continue
    # ... 计算 IQR 和 Z-score 离群值 ...
```

小贴士： `if len(data) < 50: continue` 是一个保护性判断。当样本量太小时，分位数和标准差的估计都不可靠，离群值检测没有意义。50 是一个常用的经验阈值。

2. IQR 方法（四分位距法）

2.1 原理详解

IQR（InterQuartile Range，四分位距）方法基于数据的**分位数**，是一种非参数方法，不依赖任何分布假设。

2.1.1 关键概念：Q1、Q3、IQR

- **Q1（第一四分位数，第 25 百分位数）**：将数据从小到大排序后，有 25% 的数据小于等于 Q1。
- **Q3（第三四分位数，第 75 百分位数）**：将数据从小到大排序后，有 75% 的数据小于等于 Q3。
- **IQR（四分位距）**： $IQR = Q3 - Q1$ ，表示数据中间 50% 的跨度。

重要概念： IQR 描述了数据“主体部分”的离散程度。与极差（max - min）不同，IQR 不受极端值影响，因为 Q1 和 Q3 都是分位数，极端值无论多大或多小，都不会改变它们的位置。

2.1.2 代码实现

```
q1 = data.quantile(0.25)
q3 = data.quantile(0.75)
iqr = q3 - q1
lower_iqr = q1 - 1.5 * iqr
upper_iqr = q3 + 1.5 * iqr
```

参数解释：

- `data.quantile(0.25)`：计算第 25 百分位数（即 Q1）。参数 `0.25` 表示"有 25% 的数据小于此值"。
- `data.quantile(0.75)`：计算第 75 百分位数（即 Q3）。参数 `0.75` 表示"有 75% 的数据小于此值"。
- `iqr = q3 - q1`：四分位距，中间 50% 数据的范围。
- `lower_iqr = q1 - 1.5 * iqr`：离群值下限。
- `upper_iqr = q3 + 1.5 * iqr`：离群值上限。

2.1.3 离群值判定

```
iqr_outliers = data[(data < lower_iqr) | (data > upper_iqr)]  
iqr_pct = len(iqr_outliers) / len(data) * 100
```

布尔运算详解：

- `data < lower_iqr`：返回一个布尔 Series，标记每个值是否小于下限。
- `data > upper_iqr`：返回一个布尔 Series，标记每个值是否大于上限。
- `|`：按位或（bitwise OR）运算符，对两个布尔 Series 逐元素取"或"。注意：这里**必须用 `|` 而不能用 `or`**，因为 `or` 是 Python 的逻辑运算符，无法对 pandas Series 进行逐元素操作。

常见问题：为什么写成 `(data < lower_iqr) | (data > upper_iqr)` 而不是 `data < lower_iqr | data > upper_iqr`？

因为 `|` 的优先级高于比较运算符 `<` 和 `>`。如果不加括号，Python 会先计算 `lower_iqr | data`（这会报错），再计算比较。**必须用括号明确优先级。**

2.1.4 为什么是 1.5 倍？

重要概念：1.5 这个系数并非随意选取，而是有统计学依据的。

在正态分布假设下：

- $Q1 \approx \mu - 0.6745\sigma$
- $Q3 \approx \mu + 0.6745\sigma$
- $IQR = Q3 - Q1 \approx 1.349\sigma$
- 下限 = $Q1 - 1.5 \times IQR \approx \mu - 0.6745\sigma - 1.5 \times 1.349\sigma \approx \mu - 2.698\sigma$
- 上限 = $Q3 + 1.5 \times IQR \approx \mu + 0.6745\sigma + 1.5 \times 1.349\sigma \approx \mu + 2.698\sigma$

也就是说， **$1.5 \times IQR$ 界限大约对应 $\pm 2.7\sigma$** 。在正态分布下，超出 $\pm 2.7\sigma$ 的概率约为 0.7%，这正好是一个"罕见但不至于过严"的阈值。

- 如果用 $1.0 \times IQR$ ：界限 $\approx \pm 2.0\sigma$ ，约 4.6% 数据会被标记，太宽松。
- 如果用 $3.0 \times IQR$ ：界限 $\approx \pm 4.7\sigma$ ，几乎不会标记任何数据，太严格。
- **$1.5 \times IQR$ 是 John Tukey（箱线图发明者）提出的经验值**，在大多数场景下表现良好。

小贴士：对于特别严格的场景，Tukey 还提出了 "far out" 离群值的概念，使用 $3 \times IQR$ 作为界限，对应正态分布下约 $\pm 4.7\sigma$ 。

3. Z-score 方法（标准分法）

3.1 原理详解

Z-score（标准分）方法基于数据的**均值和标准差**，是一种参数方法，隐含假设数据近似服从正态分布。

3.1.1 Z-score 公式

$$Z = \frac{x - \mu}{\sigma}$$

其中：

- x 是单个观测值
- μ 是数据的均值 (mean)
- σ 是数据的标准差 (std)

Z-score 的含义：表示一个观测值偏离均值多少个标准差。

- $Z = 0$ ：观测值等于均值
- $Z = 1$ ：观测值比均值高 1 个标准差
- $Z = -2$ ：观测值比均值低 2 个标准差
- $|Z| > 3$ ：观测值偏离均值超过 3 个标准差，通常被认为是离群值

重要概念：Z-score 本质上是一种**标准化变换** (standardization)。变换后的数据均值为 0、标准差为 1，因此不同量纲的特征可以通过 Z-score 进行横向比较。

3.1.2 代码实现

```
z_scores = np.abs((data - data.mean()) / data.std())
z_outliers = data[z_scores > 3]
z_pct = len(z_outliers) / len(data) * 100

# Mild outliers (Z-score > 2)
mild_z_outliers = data[z_scores > 2]
mild_z_pct = len(mild_z_outliers) / len(data) * 100
```

代码详解：

- `data - data.mean()`：每个值减去均值（中心化）。
- `(...) / data.std()`：再除以标准差（缩放）。这两步合起来就是标准化。
- `np.abs(...)`：取绝对值。因为我们关心的是“偏离均值多远”，而不是方向（正偏离还是负偏离）。
`np.abs` 是 NumPy 的函数，会对数组逐元素取绝对值。
- `z_scores > 3`：返回布尔 Series，标记 $|Z| > 3$ 的点。
- `data[z_scores > 3]`：用布尔索引筛选出离群值。

3.1.3 阈值的含义：3 σ 原则

重要概念 (3 σ 原则)：在正态分布下：

- 约 **68.27%** 的数据落在 $\pm 1\sigma$ 内
- 约 **95.45%** 的数据落在 $\pm 2\sigma$ 内
- 约 **99.73%** 的数据落在 $\pm 3\sigma$ 内

也就是说， $|Z| > 3$ 的概率仅约 0.27%，是非常罕见的事件，因此常被用作离群值阈值。

本教程计算了两个阈值：

| 阈值 | 含义 | 正态分布下理论比例 | 用途 |
|-----------|-------------------------|-----------|--------|
| $ Z > 2$ | Mild outlier (温和离群值) | 约 4.55% | 提示可能异常 |
| $ Z > 3$ | Extreme outlier (极端离群值) | 约 0.27% | 强烈提示异常 |

3.1.4 为什么 Z-score 对极端值敏感？

重要概念（Z-score 的致命弱点）： Z-score 公式中用到的 μ 和 σ 本身就会被极端值拉偏。

举个例子：假设有 100 个数据点，99 个在 [0, 10] 之间，1 个是 1000。

- 没有极端值时： $\mu \approx 5$, $\sigma \approx 3$
- 有极端值时： $\mu \approx 15$, $\sigma \approx 100$

可以看到，**单个极端值就让 μ 和 σ 发生了巨大变化**。这导致：

1. μ 被拉向极端值方向
2. σ 被显著放大
3. 其他正常点的 Z-score 反而变小了
4. 极端值本身的 Z-score 也被"稀释"，可能不再超过 3

这就是为什么在**偏态分布或有极端值**的数据上，Z-score 方法的效果会大打折扣。而 IQR 方法基于分位数，**不受极端值影响**，因此在这样的场景下更稳健。

小贴士： 为了克服 Z-score 的这个弱点，可以使用**修正 Z-score (Modified Z-score)**，用中位数和 MAD (Median Absolute Deviation) 替代均值和标准差。公式为： $M_i = 0.6745 \times \frac{x - \text{median}}{\text{MAD}}$ 。

4. 两种方法对比

4.1 优缺点总结

| 维度 | IQR 方法 | Z-score 方法 |
|----------|---|-------------------------------------|
| 是否参数方法 | 非参数（基于分位数） | 参数（基于均值和标准差） |
| 分布假设 | 无假设，适用于任何分布 | 假设近似正态分布 |
| 对极端值的稳健性 | 稳健 （分位数不受极端值影响） | 不稳健 （ μ 和 σ 会被拉偏） |
| 适用分布类型 | 偏态分布、重尾分布、未知分布 | 近似正态分布 |
| 计算复杂度 | 低（只需排序求分位数） | 低（只需均值和标准差） |
| 对样本量的要求 | 小样本时 IQR 估计不稳定 | 小样本时 σ 估计不稳定 |
| 可解释性 | 直观（基于箱线图） | 直观（基于标准差倍数） |
| 阈值含义 | $1.5 \times \text{IQR} \approx \pm 2.7\sigma$ （正态下） | $ Z > 3$ 对应 0.27% 概率 |

4.2 选择建议

小贴士：

- 如果数据**明显偏态或有重尾**：优先用 **IQR**。
- 如果数据**近似正态**且样本量大：**两者都可以**，Z-score 更精细。
- 如果**不确定**：**两种方法都算一遍**，对比结果。本教程就是这么做的。
- 如果数据**严重非正态**且想用 Z-score 思路：考虑**修正 Z-score**（基于中位数和 MAD）。

5. 可视化：IQR 箱线图与 Z-score 散点图

5.1 图 4a：IQR 与 Z-score 对比

5.1.1 创建画布

```
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
```

- `1, 2`：1 行 2 列的子图布局。
- `figsize=(14, 6)`：整个画布大小为 14×6 英寸。
- `axes`：是一个包含两个子图 Axes 对象的数组，`axes[0]` 是左图，`axes[1]` 是右图。

5.1.2 数据采样

```
if 'Age' in df.columns:  
    age_data = df['Age'].dropna()  
    if len(age_data) > 50000:  
        age_data = age_data.sample(50000, random_state=42)
```

小贴士：当数据量很大时（如 20 万条），绘制散点图会非常慢且点会重叠成一片。这里用 `sample(50000, random_state=42)` 随机采样 5 万条用于可视化。`random_state=42` 保证每次运行结果一致，便于复现。

5.1.3 箱线图 (Boxplot)

```
bp = axes[0].boxplot(age_data, vert=True, patch_artist=True, widths=0.3)  
bp['boxes'][0].set_facecolor('#3498db')  
bp['fliers'][0].set_markerfacecolor('#e74c3c')  
bp['fliers'][0].set_markersize(4)
```

boxplot 参数详解：

| 参数 | 值 | 含义 |
|---------------------------|-------------------|---|
| <code>age_data</code> | 数据 | 要绘制箱线图的数据 |
| <code>vert</code> | <code>True</code> | vertical ，垂直方向绘制（箱体竖立）。若为 <code>False</code> 则水平绘制。 |
| <code>patch_artist</code> | <code>True</code> | 是否用 Patch 对象填充箱体。设为 <code>True</code> 后才能用 <code>set_facecolor</code> 给箱体上色。 |
| <code>widths</code> | <code>0.3</code> | 箱体的宽度（占坐标轴比例）。 |

bp 返回值详解：

boxplot 返回一个字典，包含箱线图各个组件：

- `bp['boxes']`：箱体（Q1 到 Q3 的矩形）
- `bp['whiskers']`：须线（从箱体延伸到 1.5×IQR 界限）
- `bp['caps']`：须线末端的横线
- `bp['medians']`：中位数线
- `bp['fliers']`：离群值点（超出须线范围的点）

重要概念（箱线图各组件含义）：

- 箱体下边：Q1（第 25 百分位数）
- 箱体上边：Q3（第 75 百分位数）
- 箱体高度：IQR = Q3 - Q1

- **箱体横线**：中位数（median）
- **下须线**：从 Q1 向下延伸到 $\min(\text{数据}, Q1 - 1.5 \times IQR)$ 范围内的最小值
- **上须线**：从 Q3 向上延伸到 $\max(\text{数据}, Q3 + 1.5 \times IQR)$ 范围内的最大值
- **须线外的点**：离群值（fliers），即超出 $1.5 \times IQR$ 界限的点

5.1.4 水平参考线（axhline）

```
axes[0].axhline(y=lower, color='orange', linestyle='--', linewidth=1.5,
                label=f'Lower bound (Q1-1.5IQR)={lower:.1f}')
axes[0].axhline(y=upper, color='red', linestyle='--', linewidth=1.5,
                label=f'Upper bound (Q3+1.5IQR)={upper:.1f}')
```

axhline 参数详解（axhline = axis horizontal line）：

| 参数 | 值 | 含义 |
|-----------|------------------|-----------------------------------|
| y | lower 或 upper | 水平线的 y 坐标位置 |
| color | 'orange' / 'red' | 线的颜色。可用颜色名或十六进制（如 '#e74c3c'） |
| linestyle | '--' | 线型。'-' 实线，'--' 虚线，':' 点线，'-.' 点划线 |
| linewidth | 1.5 | 线宽（单位：磅） |
| label | 字符串 | 图例标签，配合 legend() 显示 |

常见问题：axhline 和 hlines 有什么区别？

- axhline(y=...)：画一条**横跨整个坐标轴**的水平线，常用于标注阈值。
- hlines(y, xmin, xmax)：画一条**指定区间**的水平线，需要提供 xmin 和 xmax。
- 类似地，axvline 和 vlimes 用于垂直线。

5.1.5 Z-score 散点图

```
z_scores = np.abs((age_data - age_data.mean()) / age_data.std())

axes[1].scatter(range(len(age_data)), age_data.values, c=z_scores.values,
                cmap='viridis', alpha=0.5, s=1)
```

scatter 参数详解：

| 参数 | 值 | 含义 |
|----------------------|-----------------|---|
| range(len(age_data)) | x 坐标 | 样本索引 (0, 1, 2, ...)，仅用于横向铺开点 |
| age_data.values | y 坐标 | 实际的年龄值（.values 把 Series 转为 NumPy 数组） |
| c | z_scores.values | 每个点的颜色映射依据 。这里用 Z-score 的绝对值决定颜色深浅 |
| cmap | 'viridis' | colormap ，颜色映射表。viridis 是 matplotlib 默认的感知均匀色图（从深紫到亮黄） |
| alpha | 0.5 | 透明度，0 完全透明，1 完全不透明。点密集时调低 alpha 可避免重叠遮挡 |
| s | 1 | 点的大小（单位：磅 ² ）。注意是 面积的平方 ，所以 s=1 是很小的点 |

小贴士：c 参数非常强大。它可以接受：

- 单一颜色字符串（如 'red'）：所有点同色

- 颜色列表：每个点不同色
- 数值数组：根据数值映射到 colormap 上的颜色（本例就是这种用法）

5.1.6 颜色条 (colorbar)

```
cbar = plt.colorbar(axes[1].collections[0], ax=axes[1], label='|Z-score|')
```

colorbar 详解：

- `axes[1].collections[0]`：获取散点图的 `PathCollection` 对象（即刚才画的散点集合）。`colorbar` 需要知道是哪个“可映射对象”（mappable）来生成颜色条。
- `ax=axes[1]`：指定颜色条附加到哪个子图。
- `label='|Z-score|'`：颜色条的标签。

重要概念：颜色条是数值到颜色的“图例”。在本图中，颜色从深到浅表示 `|Z-score|` 从小到大，这样一眼就能看出哪些点偏离均值最远（颜色最亮）。

5.1.7 Z-score 阈值线

```
axes[1].axhline(y=age_data.mean() + 3 * age_data.std(), color='red', linestyle='--',  
                linewidth=1.5, label=f' $\mu+3\sigma = \{age\_data.mean() + 3 * age\_data.std():.1f\}$ ')  
axes[1].axhline(y=age_data.mean() - 3 * age_data.std(), color='red', linestyle='--',  
                linewidth=1.5, label=f' $\mu-3\sigma = \{age\_data.mean() - 3 * age\_data.std():.1f\}$ ')
```

这两条红色虚线标出了 $\pm 3\sigma$ 的位置，超出这两条线的点就是 Z-score 方法定义的离群值。

5.1.8 结果图

IQR与Z-score对比

左图 (IQR 箱线图)：

- 蓝色箱体表示 Age 的主体分布 (Q1 到 Q3)。
- 橙色虚线是 IQR 下限，红色虚线是 IQR 上限。
- 红色小点是 IQR 方法定义的离群值（超出上下限的点）。

右图 (Z-score 散点图)：

- 每个点是一个样本，x 轴是样本索引，y 轴是年龄。
- 点的颜色表示 `|Z-score|` 的大小（深紫=小，亮黄=大）。
- 两条红色虚线是 $\pm 3\sigma$ 阈值，超出即为 Z-score 离群值。
- 颜色条显示 `|Z-score|` 的数值范围。

6. 可视化：多变量箱线图

6.1 图 4b：多变量箱线图

当我们要同时比较多个特征的离群值情况时，可以把它们的箱线图画在同一张图上。

6.1.1 数据准备

```
fig, ax = plt.subplots(figsize=(10, 6))
plot_cols = [c for c in numeric_features if c in df.columns][:5] # 最多5个
plot_data = df[plot_cols].dropna()
if len(plot_data) > 30000:
    plot_data = plot_data.sample(30000, random_state=42)
```

- `[:5]`：最多取 5 个特征，避免图过于拥挤。
- `dropna()`：删除任意特征缺失的行。注意：这里要求**所有选定特征都非缺失**，所以样本量会减少。
- `sample(30000, random_state=42)`：同样为了可视化性能进行采样。

6.1.2 宽表转长表 (melt)

```
plot_data_melted = plot_data.melt(var_name='Feature', value_name='Value')
```

重要概念（宽表 vs 长表）：

宽表 (Wide Format)：每个特征是一列，每行是一个样本。

| Index | Age | year | Code.of.Morphology |
|-------|-----|------|--------------------|
| 0 | 65 | 2010 | 80003 |
| 1 | 72 | 2012 | 81403 |

长表 (Long Format)：所有特征值堆叠在同一个 **Value** 列，用 **Feature** 列区分来源。

| Feature | Value |
|--------------------|-------|
| Age | 65 |
| Age | 72 |
| year | 2010 |
| year | 2012 |
| Code.of.Morphology | 80003 |
| Code.of.Morphology | 81403 |

melt 参数详解：

- `var_name='Feature'`：原列名变成新列的列名（即“特征名”列）。
- `value_name='Value'`：原值变成新列的列名（即“数值”列）。

小贴士：为什么 seaborn 需要“长表”？

因为 seaborn 的 `boxplot` 默认按某个分类变量（**x**）对数值（**y**）分组。长表格式下，`x='Feature'` 自然就把不同特征分开画了。这是“tidy data”（整洁数据）理念在可视化中的体现。

6.1.3 seaborn 箱线图

```
sns.boxplot(data=plot_data_melted, x='Feature', y='Value',
            hue='Feature', palette='Set2', ax=ax, legend=False)
```

sns.boxplot 参数详解：

| 参数 | 值 | 含义 |
|----------------------|-------------------------------|--|
| <code>data</code> | <code>plot_data_melted</code> | 长表格式的 DataFrame |
| <code>x</code> | <code>'Feature'</code> | x 轴变量（分类变量，决定箱体的分组） |
| <code>y</code> | <code>'Value'</code> | y 轴变量（数值变量，决定箱体的高度分布） |
| <code>hue</code> | <code>'Feature'</code> | 分组变量 ，决定颜色。这里用 <code>Feature</code> 本身，让每个特征有不同颜色 |
| <code>palette</code> | <code>'Set2'</code> | 调色板名称。 <code>Set2</code> 是 ColorBrewer 的一个柔和配色 |
| <code>ax</code> | <code>ax</code> | 指定绘制到哪个 Axes 对象 |
| <code>legend</code> | <code>False</code> | 是否显示图例。这里设为 <code>False</code> ，因为 x 轴标签已经标明了特征名，图例多余 |

重要概念（seaborn 0.13+ 的 `hue` 用法变化）：

在 **seaborn 0.13 之前**，如果想让每个箱体有不同颜色，可以直接用 `palette` 参数，不需要 `hue`。

在 **seaborn 0.13+**，官方推荐**显式使用 `hue` 参数**来分组着色，否则会触发 `FutureWarning`。同时，使用 `hue` 后会自动生成图例，如果不需要图例，要用 `legend=False` 关闭。

这是因为 seaborn 在统一其 API：**所有基于分组的着色都通过 `hue` 控制**，使行为更一致、可预测。

常见问题：为什么 `hue='Feature'` 和 `x='Feature'` 用了同一个变量？

这是一种常见技巧：当 `x` 是分类变量且我们想给每个类别上不同颜色时，把 `hue` 设为同一个变量即可。这样每个箱体会有自己的颜色，但不会产生额外的分组（因为分组维度和 `x` 维度一致）。

6.1.4 结果图

多变量箱线图

如何解读这张图：

- 每个箱体代表一个特征的分布。
- 由于不同特征的量纲差异巨大（Age 是几十，year 是几千，Code.of.Morphology 是几万），y 轴的绝对数值跨度很大。
- 这张图的主要价值在于**直观对比各特征离群值的数量和分布形态**，而不是精确比较数值大小。
- 可以看到 `Code.of.Morphology` 的离群值点（fliers）非常多，几乎覆盖了整个 y 轴范围——这与后面统计结果中它 18.67% 的离群值比例一致。

小贴士：如果想更公平地比较不同特征的离群值，可以先对每个特征做 Z-score 标准化，再画箱线图。这样所有特征都在同一尺度上。

7. 实际运行结果

下表是 4 个数值特征的离群值统计结果：

7.1 Age（年龄）

| 指标 | 值 |
|-----------|---------------|
| 样本量 | 209,443 |
| Q1 | 54 |
| Q3 | 76 |
| IQR | 22 |
| IQR 界限 | [21, 109] |
| IQR 离群值 | 3,023 (1.44%) |
| Z > 3 离群值 | 1,819 (0.87%) |
| Z > 2 离群值 | 8,886 (4.24%) |

解读：

- 大多数患者年龄在 54-76 岁之间，符合癌症高发于中老年的医学常识。
- IQR 下限 21 岁意味着 21 岁以下的患者被标记为离群值（详见后文讨论）。
- IQR 方法（1.44%）比 Z-score >3（0.87%）标记了更多离群值，这是因为 Age 分布略偏左（高龄端有长尾）。

7.2 Code.Profession（职业编码）

| 指标 | 值 |
|-----------|------------------|
| 样本量 | 193,966 |
| Q1 | 0 |
| Q3 | 513 |
| IQR | 513 |
| IQR 界限 | [-769.5, 1282.5] |
| IQR 离群值 | 0 (0.00%) |
| Z > 3 离群值 | 0 (0.00%) |
| Z > 2 离群值 | 6,099 (3.14%) |

解读：

- Q1=0 说明至少 25% 的样本职业编码为 0（可能是"未分类"或"无业"）。
- IQR 界限很宽（[-769.5, 1282.5]），且下限为负数（职业编码不可能为负），说明分布高度集中且偏态。
- IQR 和 Z>3 都没有标记任何离群值，但 Z>2 标记了 3.14%——这说明该特征分布非常集中，少数较高编码的点被 Z>2 捕获。

7.3 Code.of.Morphology（形态学编码）

| 指标 | 值 |
|-----------|-----------------|
| 样本量 | 209,758 |
| Q1 | 80003 |
| Q3 | 81403 |
| IQR | 1400 |
| IQR 界限 | [77903, 83503] |
| IQR 离群值 | 39,170 (18.67%) |
| Z > 3 离群值 | 13,034 (6.21%) |
| Z > 2 离群值 | 15,216 (7.25%) |

解读：

- **这是离群值比例最高的特征**，IQR 方法标记了将近 1/5 的样本！
- 形态学编码是 ICD-O-3 分类码，本质上是**分类变量**而非连续变量。把它当数值处理会导致离群值分析失去意义。
- 这说明：**离群值检测只对真正的连续型数值变量有意义**。对于编码类变量，应该用频数分析而非离群值分析。

重要概念：在 EDA 中，"数值型"不等于"连续型"。编码、ID、年份等虽然是数字，但本质是分类变量。对它们做均值、标准差、离群值检测时要格外谨慎。

7.4 year（确诊年份）

| 指标 | 值 |
|-----------|------------------|
| 样本量 | 209,758 |
| Q1 | 2010 |
| Q3 | 2015 |
| IQR | 5 |
| IQR 界限 | [2002.5, 2022.5] |
| IQR 离群值 | 6,963 (3.32%) |
| Z > 3 离群值 | 2,193 (1.05%) |
| Z > 2 离群值 | 12,407 (5.91%) |

解读：

- 大部分数据集中在 2010-2015 年。
- IQR 界限 [2002.5, 2022.5] 意味着 2002 年前或 2022 年后的记录被标记为离群值。
- 这些"离群值"很可能是真实的——只是数据收集早期或晚期的记录较少。
- year 同样是**离散型时间变量**，离群值分析的意义有限。

7.5 汇总对比表

| 特征 | 样本量 | IQR 离群值% | Z>3 离群值% | Z>2 离群值% |
|--------------------|---------|----------|----------|----------|
| Age | 209,443 | 1.44% | 0.87% | 4.24% |
| Code.Profession | 193,966 | 0.00% | 0.00% | 3.14% |
| Code.of.Morphology | 209,758 | 18.67% | 6.21% | 7.25% |
| year | 209,758 | 3.32% | 1.05% | 5.91% |

关键观察：

1. **IQR 方法通常比 $Z>3$ 标记更多离群值**（除了 Code.Profession 这种极端集中分布）。这是因为 IQR 界限 $\approx \pm 2.7\sigma$ ，比 $\pm 3\sigma$ 更宽松。
2. **Code.of.Morphology 的离群值比例异常高**（18.67%），提示它可能是分类变量被误当数值处理。
3. **Code.Profession 的 IQR 和 $Z>3$ 都是 0**，说明它的分布极度集中，IQR 界限被拉得很宽。

8. 讨论：医学异常值是否代表错误数据？

重要概念：离群值 $\neq$ 错误值。在医学数据中，“异常”往往有临床意义，不能简单地删除。

8.1 医学异常值的三种可能

可能性 1：真实的极端病例

医学数据天然存在极端值。例如：

- **极高龄患者**：100 岁以上的癌症患者虽然罕见，但确实存在。
- **罕见病理类型**：某些形态学编码对应的癌症类型本身就很少见。
- **早发癌症**：20 岁以下的癌症患者虽少，但在儿童肿瘤学中是重要研究对象。

这些“离群值”是**真实的临床信号**，删除它们会丢失重要信息。

可能性 2：特殊人群

某些离群值代表**有研究价值的特殊群体**：

- **少儿癌症患者**（age=0 可能代表新生儿或婴幼儿）：儿童癌症的病因、治疗、预后与成人截然不同，是独立的研究领域。
- **特定职业群体**：某些职业编码可能对应高风险行业（如化工、辐射相关），这些群体值得专门研究。
- **罕见年份的记录**：早期数据可能反映历史诊疗水平，晚期数据可能反映新诊疗技术的引入。

可能性 3：数据录入错误

当然，部分离群值确实是**数据质量问题**：

- **Age 录入错误**：如把 65 岁误录为 655 或 6.5。
- **编码错位**：如把形态学编码填到了职业编码字段。
- **单位错误**：如把月份当成年份。
- **缺失值填充错误**：如用 0 或 999 填充缺失值。

8.2 如何判断？

小贴士：判断医学离群值是否为错误，可以遵循以下流程：

1. **检查取值范围**：Age 是否在 $[0, 120]$ 内？year 是否在合理范围内？
2. **结合领域知识**：100 岁癌症患者合理吗？（合理）655 岁合理吗？（不合理）
3. **交叉验证**：如果 Age=0，是否有对应的儿童医院记录？如果 Age=100，是否有完整诊疗记录？
4. **查看原始数据**：检查相邻字段是否有错位痕迹。
5. **与数据提供方确认**：对于无法判断的极端值，咨询数据来源机构。

8.3 结论

结论：离群值 $\neq$ 错误值，需结合临床背景判断。在医学 EDA 中，**保留离群值并记录**通常比直接删除更稳妥。如果决定处理，应该：

- 记录处理理由（为什么删？删了多少？）
- 对比处理前后的模型表现
- 在报告中透明披露

9. 讨论：Age 的 IQR 下限是 21 岁合理吗？

9.1 问题陈述

从统计结果看，Age 的 IQR 下限是 **21 岁**。这意味着 **21 岁以下的癌症患者被 IQR 方法标记为"离群值"**。这合理吗？

9.2 统计学角度

从纯统计学角度，这是**合理的**。因为：

- 癌症确实是**老年高发**疾病，绝大多数患者年龄在 50 岁以上。
- 21 岁以下的患者在统计学上确实属于"少数尾部"，被分位数方法捕获是正常的。
- IQR 方法只关心"是否偏离主体分布"，不关心偏离的原因。

9.3 临床角度

但从临床角度，这是**不合理的**。因为：

- **儿童和青少年癌症是一个独立的医学领域**（Pediatric Oncology），有专门的诊疗指南。
- 把儿童癌症患者标记为"离群值"暗示他们是"异常"，这在伦理和科学上都有问题。
- 这些患者不仅不是"错误"，反而是**特别需要关注的研究对象**。

9.4 启示

重要概念：统计方法给出的是"统计离群值"，而非"临床离群值"。两者不能混为一谈。

这个例子生动地说明：

1. **统计工具是盲目的**：IQR 不知道"21 岁"在医学上有特殊含义。
2. **领域知识是必要的**：只有结合医学知识，才能正确解读统计结果。
3. **EDA 的目标是理解数据，而非机械地清洗数据**：发现 21 岁以下患者被标记为离群值，应该引发思考"为什么？"，而不是"删掉它们"。

小贴士：在实际项目中，遇到这种"统计离群值"与"领域知识"冲突时，推荐做法是：

- **保留这些数据**，但在分析中单独标注。
- 可以做**敏感性分析**：分别用包含/不包含这些"离群值"的数据建模，看结论是否稳定。
- 在论文或报告中**明确讨论**这一点，体现研究的严谨性。

10. 常见问题（FAQ）

Q1：为什么我的 IQR 离群值比例和 Z-score 离群值比例差很多？

A：这通常是因为数据**偏离正态分布**。在正态分布下，IQR 界限 $\approx \pm 2.7\sigma$ ，介于 $Z>2$ 和 $Z>3$ 之间。如果数据偏态或重尾，两种方法的差异会放大。建议同时看两种方法的结果，并检查数据的分布形态（直方图、QQ 图）。

Q2：离群值应该删除吗？

A：**不要盲目删除**。删除前要回答三个问题：

1. 这个离群值是错误吗？（如果是，删）

2. 删除后分析结论会变吗？（做敏感性分析）
3. 删除的理由能向同行解释清楚吗？（如果不能，别删）

Q3: | 和 or 有什么区别？

A:

- | 是按位或运算符，可以对 pandas Series / NumPy 数组进行逐元素运算，返回布尔数组。
- or 是 Python 的逻辑或运算符，只能对单个布尔值运算，作用于 Series 时会报错（`ValueError: The truth value of a Series is ambiguous`）。
- 类似地，& 对应 and，~ 对应 not。在 pandas 中过滤数据时，必须用 &、|、~，且要用括号分组。

Q4: 为什么箱线图的"须线"不等于 $Q1 - 1.5 \times IQR$ ？

A: 这是一个常见误解。箱线图的须线不是直接画在 $Q1 - 1.5 \times IQR$ 处，而是画在实际数据中不超过该界限的最值处。也就是说：

- 下须线 = $\max(\min(\text{data}), Q1 - 1.5 \times IQR)$ 范围内的最小实际数据值
- 上须线 = $\min(\max(\text{data}), Q3 + 1.5 \times IQR)$ 范围内的最大实际数据值

所以须线总是落在某个真实数据点上，而不是界限本身。界限只是用来判定哪些点是 fliers。

Q5: seaborn 报 `FutureWarning: Setting a gradient palette...` 怎么办？

A: 这是 seaborn 0.13+ 的提示。解决方法是显式使用 `hue` 参数，并设置 `legend=False`（如果不需要图例）。本教程的代码就是这么写的：

```
sns.boxplot(data=..., x='Feature', y='Value', hue='Feature', palette='Set2', legend=False)
```

Q6: 为什么 `Code.of.Morphology` 的离群值比例高达 18.67%？

A: 因为它是分类编码而非连续变量。形态学编码是 ICD-O-3 体系的标识码（如 80003 代表"上皮细胞肿瘤"），数字大小没有数值意义。把它当连续变量算 IQR 和 Z-score 本身就是误用。正确做法是用 `value_counts()` 看频数分布，或将其作为分类变量处理。

11. 模块总结

本模块学到的知识点

理论知识

1. IQR 方法：

- $Q1$ （第 25 百分位数）、 $Q3$ （第 75 百分位数）、 $IQR = Q3 - Q1$ 的含义
- 离群值界限： $Q1 - 1.5 \times IQR$ 和 $Q3 + 1.5 \times IQR$
- 1.5 倍系数的统计学依据（正态分布下约对应 $\pm 2.7\sigma$ ）
- IQR 方法是**非参数**方法，对极端值**稳健**

2. Z-score 方法：

- 公式 $Z = (x - \mu) / \sigma$ 的含义（标准化变换）
- 3σ 原则（正态分布下 99.7% 数据在 $\pm 3\sigma$ 内）
- $Z > 3$ （极端离群值）和 $Z > 2$ （温和离群值）的阈值含义
- Z-score 方法的**致命弱点**： μ 和 σ 会被极端值拉偏

3. 两种方法对比：

- IQR 适用于偏态分布、重尾分布、未知分布
- Z-score 适用于近似正态分布
- 不确定时**两种都算一遍**，对比结果

代码技能

1. pandas 操作:

- `data.quantile(0.25) / data.quantile(0.75)` 计算分位数
- 布尔索引 `(data < lower) | (data > upper)`，注意 `|` 是按位或，**必须用括号分组**
- `df.melt(var_name=..., value_name=...)` 宽表转长表

2. NumPy 操作:

- `np.abs(...)` 取绝对值
- 标准化变换 `(data - data.mean()) / data.std()`

3. matplotlib 可视化:

- `plt.subplots(1, 2, figsize=...)` 创建多子图画布
- `axes[0].boxplot(data, vert=True, patch_artist=True, widths=0.3)` 绘制箱线图
- `axes[0].axhline(y=..., color=..., linestyle='--', linewidth=...)` 画水平参考线
- `axes[1].scatter(x, y, c=..., cmap='viridis', alpha=0.5, s=1)` 绘制带颜色映射的散点图
- `plt.colorbar(mappable, ax=..., label=...)` 添加颜色条

4. seaborn 可视化:

- `sns.boxplot(data=..., x=..., y=..., hue=..., palette=..., ax=..., legend=False)`
- seaborn 0.13+ 的 `hue` 用法变化及 `legend=False` 的作用

思维方法

1. 批判性思考:

- 离群值 $\neq$ 错误值，需结合领域知识判断
- 医学异常值的三种可能：真实极端病例、特殊人群、录入错误
- 统计离群值与临床离群值的区别（Age 21 岁下限的例子）
- "数值型"不等于"连续型"，编码类变量不应做离群值分析

2. 最佳实践:

- 大数据集可视化前先采样（`sample(50000, random_state=42)`）
- 不确定分布时多种方法并用
- 保留离群值并记录处理理由，而非盲目删除
- 做敏感性分析，对比处理前后的结论稳定性

下一模块预告

在下一个模块中，我们将进入**统计检验**的世界，学习如何用 p 值、效应量等方法量化特征与目标变量之间的关联，为特征选择打下基础。

最后提醒：离群值分析是 EDA 中"发现问题"的工具，而非"解决问题"的工具。发现离群值后，**理解它们为什么存在**比"如何处理它们"更重要。保持好奇心和批判性思维，是数据科学家的核心素养。